

From Search Dynamics to Semi-Ideal Computers

Theorem Mazes, Discovery Coordinates, and Feedback Control for Automated Reasoning

Brian Tenneson

August 14, 2026 – Version 0.4

Suggested arXiv classifications: primary `cs.LG`; secondary `cs.AI`; cross-list candidate `math.CO`.

Abstract

Automated theorem proving can be viewed as navigation through an implicit directed maze: proof states are vertices, legal inference steps are edges, and verifier-accepted certificates are absorbing exits. This viewpoint separates two problems that are often conflated. Shortest-path algorithms are efficient on many explicit graphs, but a compact logical problem may induce an enormous proof-state graph. The practical objective is therefore not merely to choose a path well after the maze is exposed, but to expose as little of the maze as possible while preserving certificate correctness.

The paper develops this program in three layers. First, it prioritizes exact or conservative reductions—including quotienting, partial-order reduction, admissible abstractions, verified landmarks, proof-horizon bounds, and related graph reductions—whenever their hypotheses apply. Second, when exact reduction is insufficient, it treats automated reasoning as a feedback-control problem in which learned quantities such as drift, variance, Bellman values, hazard, motif scores, and resource allocations guide search but never replace the independent verifier. Third, it formalizes discovery as a verifier-backed event and introduces graded discovery-state quantities for experiments in which a binary success/failure endpoint is too coarse.

A central mathematical connection in Version 0.4 is the Bellman–Abel correspondence for discovery. If an optimal Bellman value represents expected time-to-discovery and $A^* = -V^*$, then before discovery the optimal recursion induces an Abel-type translation law; with nonnegative rational action costs, the translation is cost adjusted. Approximate Abel coordinates, Bellman–Abel residuals, drift, variance, and action-gap stability then provide measurable signals for successor selection and experimental diagnosis.

The empirical methodology is deliberately forward chained and certificate first. Historical solutions are hidden at evaluation time, models are trained only on earlier audited rungs, and the rule is *learn between experiments; freeze within experiments*. The first strict nine-rung H2 study is retained as a negative control: under a matched budget of $R = 8$ verifier calls per target per action, all five intervention families reached the same two of nine rungs. Thus the current paper makes no end-to-end ATP speedup claim. Instead, that result identifies a measurement limitation of binary discovery gain and motivates the next transition-level experiment using graded Abel-state logging.

Semi-ideal computers, ATPs, automated logical deciders, automated metalogical deciders, reflective search, absorbing fixed points, semigroup and IFS descriptions, and DATA as a distinguished implementation are developed as compatible layers of the same certificate-preserving framework. Their role is to enlarge the control vocabulary without changing the authority condition: settlement is accepted only when an independent verifier accepts the corresponding certificate.

Contents

1	Big picture: shrink the theorem maze before trying to outrun it	5
1.1	The program in one diagram	5
1.2	Most promising theories	5
1.3	The strongest near-term synthesis: quotient, abstraction, and bounds	5
1.4	A natural algebra for proof classes: min-plus	7
1.5	What should be tested first	7
2	Maze theory first: the minimum needed for theorem search	8
2.1	Mazes as directed graphs	8
2.2	Unweighted and weighted mazes	8
2.3	Effective naming of rational edge weights	9
2.4	What “escaping the maze” normally costs	9
2.5	Implicit mazes and the usual exponential search bound	9
2.6	From an ordinary maze to a theorem maze	10
2.7	Unit normalization and the BFS baseline	11
2.8	Rational weights are costs, not fractional inferences	11
2.9	Where geometry and Hilbert ordering enter	12
2.10	The big picture before the rest of the paper	12
3	Theorem proving as rationally weighted maze solving	12
4	Search-space reduction before learning	13
4.1	Cumulative verified-history collapse	13
4.2	Novelty DAG	14
4.3	Quotienting by verified equivalence	14
4.4	Conservative compilation and macro edges	14
4.5	Shared verified landmarks	14
4.6	Incumbent bounds	14
5	Heuristic branches and where to look	15
6	The theoremhood quotient: proofs as representatives	15
7	Purpose and scope	16
8	General verified transition systems	17
9	The cumulative history of verified information	17
10	The cumulative-history SIC theorem	18
11	When the induced SIC is an ATP, ALD, or AMLD	18
11.1	ATP specialization	18
11.2	ALD specialization	19
11.3	AMLD specialization	19
12	Halting and absorbing fixed points	19

13 Control AMLD theory: from static compassing to feedback settlement	19
13.1 State, actions, and the control objective	20
13.2 Abel coordinates and control-Lyapunov coordinates	20
13.3 P/R/I control and resource allocation	21
13.4 Five deliberately open control-AMLD conjectures	22
14 Our settlement-compass experiments	22
14.1 Experiment 004: frozen nested shells and the MAX “stupid case”	22
14.2 Illustrative learning-curve fits	23
14.3 Experiment 005: the controller is harder than the compass	24
14.4 What the experiments currently support	24
14.5 Next control experiments	25
15 Prior synthetic benchmark evidence, with disclaimers	25
16 A proposed rational-weight theorem-maze benchmark	26
17 Reflective ATPs: proving things about the prover	26
18 DATA as a distinguished case	27
19 A search funnel for computational accessibility	28
20 Research conjectures and implementation questions	28
21 Discovery	29
21.1 Weak and operational discovery	29
21.2 Binary event, continuous precursors	30
21.3 Discovery hazard and discovery depth	31
21.4 Discovery gain as a control quantity	31
21.5 Retrospective controls on solved rungs	32
21.6 Forward-chaining rung by rung	32
21.7 Motifs, structured variation, and the music analogy	33
21.8 Relation to SICs, ALDs, AMLDs, and fixed points	33
21.9 What would count as evidence	34
22 The Abel equation and approximate discovery coordinates	34
22.1 The discovery set and the pre-terminal Abel law	34
22.2 Abel drift, Bellman-Abel residual, and action scores	36
22.3 Abel discovery advantage versus binary discovery gain	36
22.4 Approximation and action-gap stability	37
22.5 Positive Abel drift and discovery time	37
22.6 Learning an approximate Abel coordinate without leakage	38
23 Experimental design for iterative discovery control	38
23.1 Three statistical units: rung, trajectory, transition	39
23.2 The frozen forward-chain protocol	39
23.3 The five-arm matched comparison	40
23.4 Primary and secondary measurements	40

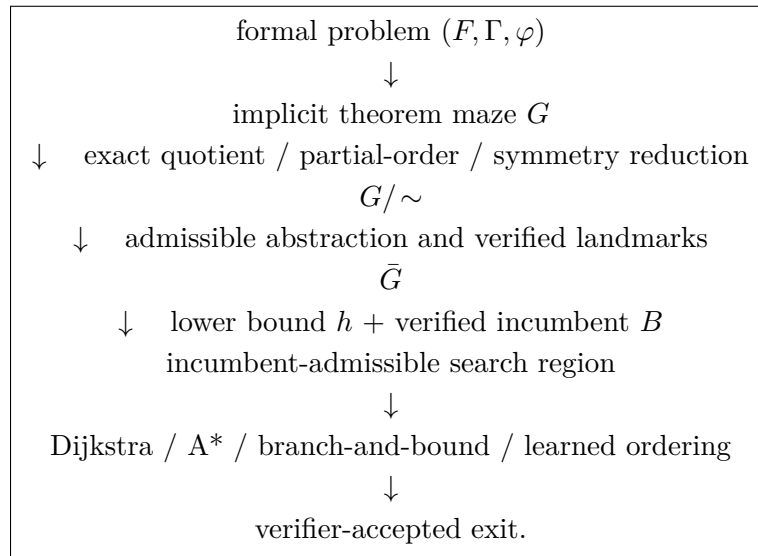
23.5 Iteration 1: the nine-rung H2 forward chain	41
23.6 Iteration 2: Discovery Abel v4	42
23.7 Analysis rules for small forward chains	42
23.8 Escalation across theorem families	43
23.9 Decision rules for the next iteration	43
24 Conclusion	43

1 Big picture: shrink the theorem maze before trying to outrun it

Research judgment, not a theorem. The ranking in this section is a research-priority assessment. It is based on which ideas attack the explicit bottleneck identified by the maze formulation—state-space explosion—while preserving exact logical correctness. Classical graph search already solves an explicitly presented finite maze efficiently relative to the size of that maze. The hard ATP problem is that the theorem maze is normally implicit and may be enormous before a useful exit is exposed. The most promising theories are therefore the ones that can provably or measurably reduce the state space, supply exact lower or upper bounds, or reuse verified structure before search expands most of the maze [14, 15, 16].

1.1 The program in one diagram

The proposed near-term hierarchy is



The central aim is not to make every proof short. It is to arrange that the machine never materializes large families of routes that are redundant for the declared objective.

1.2 Most promising theories

1.3 The strongest near-term synthesis: quotient, abstraction, and bounds

The most concrete mathematical target is to combine exact state reduction with an admissible abstract distance. Let

$$G = (V, E, s, Z, w)$$

be a nonnegatively weighted theorem maze. First choose a verifier-respecting equivalence relation $\sim$ that merges only states whose distinction is irrelevant to the reachability and cost objective being preserved. Standard symmetry and partial-order reductions show how dramatic such reductions can be in other reachability systems, but the proof-specific preservation conditions must be proved rather than assumed [8, 9, 10].

Next let

$$\pi : V \rightarrow \bar{V}$$

Priority	Theory or method	Why it looks promising for ATP search
1	Quotient state spaces, symmetry reduction, partial-order reduction, proof-DAG canonicalization	Directly attacks state explosion by identifying histories or interleavings that need not be explored separately. Model checking developed quotient, symmetry, and partial-order methods precisely because explicit reachability graphs explode; the proof-search analogue is to preserve theorem reachability and the chosen cost while exploring fewer representatives [8, 9, 10, 14].
2	Admissible abstraction + A* / branch-and-bound + Proof Horizon	Gives a mathematically safe way to combine a lower bound on cost-to-go with an upper bound from any verified incumbent. This can remove whole regions without trusting a learned guess. Pattern-database and abstraction heuristics show how abstract state spaces can generate admissible lower bounds in other search domains [3, 11, 15, 16].
3	Conservative compilation, verified shared landmarks, dynamic programming on proof DAGs	Reuses repeated verified substructure instead of rediscovering it. This is especially attractive when many routes share the same lemmas or subproofs; the author’s earlier compilation and bank results give theorem-search-specific motivation [14, 19].
4	Min-plus / tropical path algebra over exact rational costs	Gives a clean algebraic home for weighted theorem mazes: sequential inference costs add and alternative proof routes are minimized. It is more a unifying theory than a guaranteed speedup, but it may make dynamic programming, composition, and quotient costs easier to state and analyze [12].
5	Learned cost-to-go, portfolio scheduling, Hilbert/locality ordering	Potentially powerful for deciding which surviving state to expand first. These methods can improve navigation without changing the verifier, but their advantage is empirical and distribution-dependent; Hilbert ordering in particular should be treated as a benchmarked locality heuristic rather than an optimality theorem [18, 19].
6	IFS/semigroup, fractional iteration, reflective meta-ATP	Mathematically interesting long-term control theories. They may eventually describe hitting-time geometry or prove bounded facts about the searcher, but they are not prerequisites for the maze reduction program and currently have a less direct path to speed than exact quotienting and admissible bounds [19, 14].

Table 1: Research-priority assessment for the maze-first program. The ordering is a judgment about likely leverage, not a proved dominance theorem.

map the concrete maze into an abstract or relaxed maze $\bar{G} = (\bar{V}, \bar{E}, \bar{Z}, \bar{w})$. Assume that every concrete path from v to an exit maps to an abstract path from $\pi(v)$ to $\bar{Z}$ of no greater cost.

Proposition 1.1 (Abstract-distance lower bound). *Under the preceding path-relaxation hypothesis,*

$$h(v) := d_{\bar{w}}(\pi(v), \bar{Z})$$

is admissible for the concrete theorem maze:

$$h(v) \leq d_w(v, Z).$$

Proof. Every concrete v -to-exit path has an abstract image of no greater cost. Taking the minimum or infimum over abstract paths therefore cannot exceed the minimum or infimum over concrete paths. $\square$

Now suppose the fast search wing has already found and verified an exit route of cost B . For a reached state v , let $g(v)$ be the exact cost already spent. Any route improving the incumbent must

satisfy

$$g(v) + h(v) < B.$$

Therefore the next search can be confined to the *incumbent-admissible region*

$$\mathcal{C}_B(h) = \{v \in V : g(v) + h(v) < B\}.$$

This is the clearest current mathematical answer to the question “where should the ATP not look?” The incumbent supplies an exact upper bound; the abstraction supplies an exact lower bound; quotienting reduces duplicate states before either bound is applied. The tighter the valid quotient and the lower bound, the smaller the region that remains eligible for an improving proof [3, 11, 15].

Caution 1.2 (Keep the cost metric fixed). The same metric must underlie g , h , and B . A native proof-length Horizon cannot be combined without proof with a heuristic that lower-bounds wall-clock search cost, and a macro-edge search cost need not equal the cost of its transparent native expansion. This metric-separation issue is already explicit in the DATA/Horizon framework [19].

1.4 A natural algebra for proof classes: min-plus

The weighted maze and the proof-tour quotient fit the min-plus (tropical) semiring

$$\mathbb{T}_{\mathbb{Q}} = (\mathbb{Q}_{\geq 0} \cup \{\infty\}, \oplus, \otimes), \quad a \oplus b = \min(a, b), \quad a \otimes b = a + b.$$

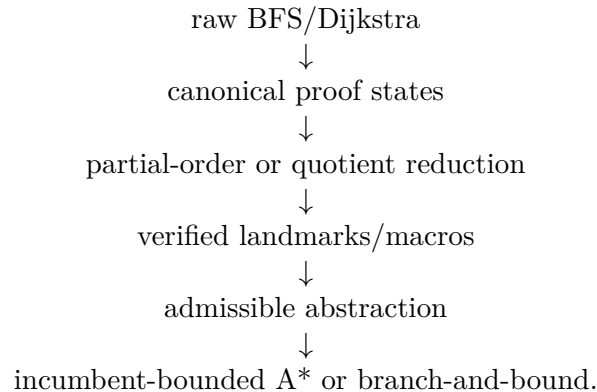
Along one proof route, edge costs compose by $+$; among alternative routes proving the same theorem, the optimal representative is selected by $\min$. On a finite or otherwise effectively proper layer where the minimum is attained, the weighted Proof Horizon can therefore be written schematically as

$$H_w(\Gamma, \varphi) = \bigoplus_{P \in [\varphi]} \bigotimes_{e \in P} w(e).$$

This does not make state explosion disappear. Its value is structural: it turns “proofs are paths and the Horizon is the cheapest representative” into an ordinary algebraic-path statement, connecting the theorem-maze program to the established theory of graphs over semirings and dioids [12].

1.5 What should be tested first

The first serious implementation target should therefore be a matched benchmark in which the same verifier-backed theorem mazes are searched under progressively stronger but separately measured reductions:



Learned ranking and Hilbert ordering can then be layered on the remaining region and evaluated by ablation. Fractional/continuous dynamics should be tested later, after the simpler graph reductions have been exhausted. This ordering makes failures informative: if a speedup occurs, the experiment can identify whether it came from structural compression, exact bounds, or heuristic navigation rather than attributing everything to one large architecture.

2 Maze theory first: the minimum needed for theorem search

The main organizing idea of this sequel is deliberately elementary: before discussing SICs, fixed points, IFS language, or reflective control, treat theorem proving as maze solving. A maze is a graph-search problem. The classical shortest-path literature then tells us what can be solved efficiently once the graph is explicitly available, while the theorem-proving difficulty is pushed into the size and structure of the implicit graph that must be generated. Moore’s early maze formulation, Dijkstra’s weighted shortest-path algorithm, and the later A* theory are the basic classical reference points [1, 2, 3, 4].

2.1 Mazes as directed graphs

For present purposes a maze is a directed graph with an entrance and one or more exits,

$$\mathcal{M} = (V, E, s, Z),$$

where V is the vertex set, $E \subseteq V \times V$ is the directed edge set, $s \in V$ is the entrance, and $Z \subseteq V$ is the exit set. An edge $u \rightarrow v$ means that moving from u to v is legal. A finite directed walk is a sequence of legal edges; a path is a walk with no repeated vertices. In this manuscript the informal word “tour” means a finite directed walk from the entrance toward an exit, not necessarily a closed tour.

A graph is *sparse* when $|E|$ is of the same order as $|V|$ and *dense* when $|E| = \Theta(|V|^2)$. This distinction matters because an $O(|V| + |E|)$ algorithm is linear in the explicit graph representation but becomes $O(|V|^2)$ on a dense maze [4].

2.2 Unweighted and weighted mazes

In an unweighted maze every edge may be assigned unit cost,

$$w(e) = 1.$$

Then the cost of a path is simply its number of edges. Breadth-first search (BFS) therefore finds a route with the minimum number of edges from the entrance to an exit [1, 4].

A weighted maze adds a cost map

$$w : E \rightarrow \mathbb{R}.$$

This paper restricts the exact ATP cost layer to

$$w : E \rightarrow \mathbb{Q}_{\geq 0}.$$

The restriction to exact nonnegative rational numbers is intentional. Rational weights can be represented by finite numerator/denominator pairs, added exactly, and compared exactly. Thus an optimality statement need not depend on floating-point roundoff. Dijkstra’s algorithm applies to nonnegative weights, and A* adds an admissible estimate of the remaining cost [2, 3, 4].

The path cost is

$$w(P) = \sum_{e \in P} w(e).$$

The shortest escape cost is

$$d_w(s, Z) = \inf\{w(P) : P \text{ is a directed path from } s \text{ to some } z \in Z\}.$$

For a finite maze with nonnegative weights and a reachable exit, the infimum is attained by a simple path.

2.3 Effective naming of rational edge weights

The author’s earlier closed-form surjection

$$r : \mathbb{N} \twoheadrightarrow \mathbb{Q}$$

provides an explicit natural-number naming scheme for rational values [13]. Taking

$$r_+(n) = |r(n)|$$

gives a surjection $r_+ : \mathbb{N} \twoheadrightarrow \mathbb{Q}_{\geq 0}$. Hence a theorem-maze edge may store a natural code k_e while its exact declared cost is

$$w(e) = r_+(k_e).$$

This coding is not needed for Dijkstra or A*—ordinary reduced numerator/denominator pairs already suffice—but it is useful conceptually and computationally when one wants every exact rational cost to have a discrete address. It also keeps two roles separate: rational codes name edge costs, while Hilbert-style coordinates and orderings may later be used to name or prioritize regions of the state space [18].

2.4 What “escaping the maze” normally costs

Let $n = |V|$ and $m = |E|$. The following are the standard worst-case graph-operation bounds needed later. They describe an *explicitly represented* finite maze. Exact rational arithmetic adds the cost of exact integer addition and comparison; the table suppresses that bit-complexity factor and records the usual graph-search bounds [4, 2, 3, 6].

The table gives an important perspective. Rational edge weights do not by themselves make shortest-path search intractable. For a dense finite maze that has already been written down, BFS, Dijkstra with a dense implementation, and DAG shortest-path dynamic programming all have familiar $O(n^2)$ graph-operation bounds. The difficulty for ATP is that n itself may be exponential or worse as a function of a compact formal problem description. A procedure polynomial in the number of explicit proof states can therefore still be impractical in the original theorem-proving input size. This explicit-versus-implicit distinction is central to the search-reduction program developed below.

2.5 Implicit mazes and the usual exponential search bound

The same point is often expressed in AI search notation. Suppose the maze is not given in advance but is generated as a search tree with effective branching factor b , the shallowest exit is at depth d , and a depth-first search may descend to depth ℓ . In the usual tree-search accounting, BFS can require $O(b^d)$ generated nodes and $O(b^d)$ memory; DFS can require $O(b^\ell)$ time while using only linear-in-depth stack memory up to the cost of storing successor information. A* has no

Method	Guarantee	General explicit-graph worst case	Dense maze $m = \Theta(n^2)$
DFS	Finds some reachable exit; not generally shortest	$O(n + m)$	$O(n^2)$
BFS	Minimum number of edges when all costs are 1	$O(n + m)$	$O(n^2)$
Dijkstra	Minimum total cost for nonnegative weights	$O((n + m) \log n)$ with a binary heap; $O(n^2)$ with a simple array implementation	$O(n^2)$ is standard and natural for dense graphs
A*	Minimum cost under the usual admissibility/implementation hypotheses	Worst case may search essentially the same region as Dijkstra; $h = 0$ reduces to Dijkstra	No worst-case asymptotic miracle follows from heuristic search alone
DAG shortest path	Minimum total cost after a topological ordering	$O(n + m)$	$O(n^2)$
Floyd–Warshall	All-pairs shortest-path distances	$O(n^3)$	$O(n^3)$

Table 2: Usual worst-case escape/shortest-path bounds for an explicit finite maze. The ATP difficulty discussed below is that the theorem maze is normally implicit and can be enormously larger than its logical input description.

general worst-case polynomial-in- d guarantee: with an uninformative heuristic it degenerates toward uniform-cost/Dijkstra-style expansion, and standard worst cases remain exponential in solution depth [5, 3]. These bounds are not in conflict with the $O(n + m)$ or $O(n^2)$ explicit-graph bounds above: in an implicit maze, the number n of states that would have been written down can itself be on the order of b^d .

For theorem proving this is the more revealing worst case. The classical graph algorithm may be polynomial in the maze it sees, while generating enough of that maze to reach the theorem can still consume exponential resources. The central design problem is therefore to reduce the effective branching factor, identify equivalences, reuse landmarks, improve lower bounds, or otherwise avoid materializing most of the search tree.

2.6 From an ordinary maze to a theorem maze

Fix a formal system F , a set of hypotheses and/or available axioms Γ , and a target WFF φ . Following the theorem-graph program developed in the author’s earlier work, use *proof states* rather than individual formulas as vertices [14]. A proof state is a cumulative set p of formulas already available, with

$$\Gamma \subseteq p.$$

The entrance is

$$s = \Gamma.$$

The exit set is

$$Z_\varphi = \{p : \varphi \in p\}.$$

Thus an ATP succeeds exactly when it reaches a proof state containing the target.

The state formulation also resolves the arity issue for inference rules. Suppose r is a k -ary inference rule and

$$a_1, \dots, a_k \in p$$

are premises already present in the current state. If a legal application of r derives β , place the directed edge

$$p \xrightarrow{r(a_1, \dots, a_k)} p \cup \{\beta\}.$$

A binary or k -ary inference therefore remains a single ordinary graph edge because all premises are already contained in the source proof state. If one instead insists that vertices are individual WFFs, the natural object is a directed hypergraph or a bipartite formula/inference graph. For algorithmic purposes, the proof-state graph keeps the representation within ordinary directed graph theory [14].

A proof of φ is therefore the pullback of a successful directed route

$$\Gamma \rightsquigarrow Z_\varphi.$$

The verifier checks that every edge label is a legitimate inference application. Graph search chooses the route; logic decides whether each move is legal.

2.7 Unit normalization and the BFS baseline

The clean baseline is

$$w(e) = 1 \quad \text{for every legal inference edge.}$$

Then a proof route $P = (e_1, \dots, e_L)$ has cost

$$w(P) = \sum_{i=1}^L 1 = L.$$

Consequently the theorem-maze distance is exactly shortest proof length,

$$d_1(\Gamma, Z_\varphi) = \min\{|P| : P \text{ proves } \varphi\}.$$

This is the graph-theoretic form of the author's Proof-Horizon/BFS viewpoint: unit-weight breadth-first search reaches a target first at the minimum inference depth [14, 15]. Choosing the unit weight 1 is a normalization: one does not need to carry a separate physical unit through every formula. One edge simply means one unit of declared proof cost.

2.8 Rational weights are costs, not fractional inferences

After the unit case is understood, one may declare different exact rational costs,

$$w(e) \in \mathbb{Q}_{\geq 0}.$$

A weight such as $3/7$ means that the edge costs $3/7$ of the chosen cost unit. It does *not* by itself mean that the inference is the fractional iterate $T^{3/7}$ of a logical transformation. Fractional iteration is a separate dynamical question requiring a semigroup or embedding law such as

$$\Phi_{s+t} = \Phi_s \circ \Phi_t.$$

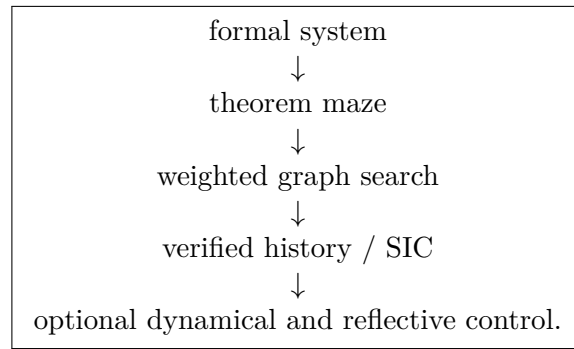
The weighted-maze layer should therefore be established independently. Continuous or fractional search dynamics may later be placed over the maze as an optional model of navigation, but rational edge length alone does not imply fractional inference.

2.9 Where geometry and Hilbert ordering enter

The author's Hilbert theorem-search work proposes using space-filling-curve orderings as a navigation device after proof/search states have been embedded into a finite-dimensional feature representation [18]. In the maze language this is best viewed as an ordering policy for which region or frontier point to inspect next. It does not alter the legal edges, the verifier, or the edge costs. The classical shortest-path guarantees remain attached to BFS, Dijkstra, A*, DAG methods, and related graph algorithms; Hilbert ordering is a candidate locality-preserving heuristic to be benchmarked against those baselines.

2.10 The big picture before the rest of the paper

The sequel will therefore use the hierarchy



The first computational question is not whether a fixed point or proof exists in the abstract, but how much of the implicit maze must be generated and searched before a verifier-accepted exit is reached. The remaining sections study exact reductions of that search region, quotient ideas, incumbent bounds, learned navigation, and only afterward the broader SIC/IFS/reflective interpretation.

3 Theorem proving as rationally weighted maze solving

The graph-theoretic viewpoint from *Depths of a Simulation* can be strengthened into a transfer principle. Proof search is treated as directed graph traversal: proof states are vertices, legal inference transactions are edges, and target states are exits [14].

Definition 3.1 (Rationally weighted theorem maze). *A rationally weighted theorem maze is*

$$\mathcal{G} = (V, E, s, Z, w)$$

where (V, E) is a directed graph, $s \in V$ is the initial proof/search state, $Z \subseteq V$ is the set of certificate-bearing exits, and

$$w : E \rightarrow \mathbb{Q}_{\geq 0}$$

assigns each directed edge an exact nonnegative rational cost. The cost of a finite path $P = (e_1, \dots, e_m)$ is

$$w(P) = \sum_{i=1}^m w(e_i) \in \mathbb{Q}_{\geq 0}.$$

Rational weights are a deliberate restriction. They can be stored, added, and compared exactly. This avoids making a logical optimality theorem depend on floating-point ordering accidents. Integer transaction counts are the special case $w(e) \in \mathbb{N}$.

Theorem 3.2 (Rational-Weighted Maze Transfer Principle). *Let $\mathcal{A}$ be a maze-search algorithm whose correctness theorem holds for a class $\mathcal{C}$ of effectively presented directed graphs with nonnegative rational edge weights. Suppose a theorem-proving or settlement problem can be encoded as a member of $\mathcal{C}$ so that:*

- (i) *vertices encode admissible proof/search states;*
- (ii) *directed edges encode admissible transactions or inference steps;*
- (iii) *target vertices are exactly states containing a verifier-accepted certificate of the requested type; and*
- (iv) *path weight is the declared ATP search cost.*

Then applying $\mathcal{A}$ to the encoded theorem maze yields an ATP/ALD/AMLD search policy with the same route-finding and path-cost guarantee, subject to the same graph-theoretic hypotheses.

Proof. The encoding is a cost-preserving correspondence between maze paths and admissible search paths. Therefore any path returned by $\mathcal{A}$ pulls back to an admissible ATP path, and any optimality guarantee over the maze path weights transfers to the declared ATP search cost. Logical acceptance still requires the independent certificate verifier. $\square$

Remark 3.3 (Examples). For finite nonnegative rationally weighted theorem mazes, Dijkstra’s algorithm gives a minimum-cost route. A* gives the same conclusion under the usual admissibility conditions, with the standard implementation qualifications concerning consistency and node reopening. On a finite DAG, dynamic programming after a topological ordering gives minimum-cost routes. These are not new graph theorems; the contribution is the exact pullback to verifier-backed proof or settlement search.

Caution 3.4 (Search cost versus native proof cost). A weighted theorem maze may price search transactions, macro rules, verifier calls, or other operations. That metric need not equal native proof length. If a banked lemma is transparently expanded before native proof cost is measured, the bank may dramatically reduce search cost while leaving the native minimum proof Horizon unchanged [19]. Every reported shortest-path claim must therefore name the metric and presentation.

4 Search-space reduction before learning

The graph interpretation provides several exact or theorem-backed reductions before any learned heuristic is trusted.

4.1 Cumulative verified-history collapse

Many transient trajectories can lead to the same permanent verified ledger. If the objective is mathematical settlement rather than replaying every heuristic choice, states that differ only in irrelevant transient history can often share downstream mathematical information. Care is required because the cumulative history alone need not determine future controller behavior, but it is already a major conceptual reduction in what must be remembered as mathematical progress.

4.2 Novelty DAG

In the novelty state theorem graph, retain only nontrivial transitions that add new verified information. Strict inclusion then rules out directed cycles, producing a DAG [14]. Identity moves, rediscovery loops, and reversible syntactic wandering can be suppressed or represented outside the verified-state graph.

4.3 Quotienting by verified equivalence

When the formal system supplies a sound equivalence relation and the search objective is invariant under it, equivalent presentations can be quotiented or canonically oriented. This is a standard graph reduction, but in ATP design it can eliminate large families of syntactic duplicates. The quotient must be verifier-respecting; heuristic similarity is not enough.

4.4 Conservative compilation and macro edges

Depths of a Simulation proves a conservative compilation result for finite provable workloads: verified derivations can be packaged as sound derived rules, lowering target transaction counts without changing theoremhood [14]. In graph language, a previously expensive verified path is replaced by a reusable macro edge. This is the mathematical justification for viewing a theorem library as a compression layer over repeated proof structure.

4.5 Shared verified landmarks

A shared bank can turn repeated rediscovery into one verified discovery followed by cheaper retrieval. DATA 3.0 gives a conditional repeated-substructure savings theorem and separately shows that transparent lemma reuse need not change the native proof Horizon [19]. For search, the bank changes the geometry; for native minimum proof cost, it need not.

4.6 Incumbent bounds

The Proof-Horizon program supplies an especially useful reduction. Once any valid proof or settlement certificate of cost B is verified, the unknown optimum c^* satisfies

$$c^* \leq B.$$

For minimum-cost certification, all candidates with cost $\geq B$ are irrelevant to the question whether a cheaper certificate exists. Each improved incumbent shrinks the remaining bounded region [15, 16].

Proposition 4.1 (Rational branch-and-bound pruning). *Let $\mathcal{G}$ be a theorem maze with nonnegative rational edge weights. Let B be the cost of a verified incumbent route to an exit. For a reached vertex v , let $g(v)$ be the exact cost already incurred and let $h(v) \in \mathbb{Q}_{\geq 0}$ be a valid lower bound on the cost of every continuation from v to an exit. If*

$$g(v) + h(v) \geq B,$$

then no route through v can improve the incumbent, so v may be pruned from a search whose sole objective is to find a route of cost $< B$.

Proof. Every continuation through v has total cost at least $g(v) + h(v) \geq B$. □

This proposition combines the Horizon incumbent idea with ordinary weighted-maze branch-and-bound. Even the trivial lower bound $h = 0$ removes every partial path whose cost has already reached B ; stronger admissible lower bounds remove more.

5 Heuristic branches and where to look

The Branch-Covering Theorem in *Depths* says, roughly, that a proof-covering nondeterministic policy contains a branch able to shadow a shortest proof; heuristics then face the navigation problem of allocating enough computation to such branches [14]. This does not identify the good branch, but it transforms the intelligence problem into ranking, covering, diversification, and resource allocation.

For a theorem maze, a learned critic can estimate quantities such as

$$\hat{d}(v, Z), \quad \hat{P}(\text{settlement within budget} \mid v), \quad \hat{Q}(v, a),$$

without ever acquiring proof authority. The verifier decides acceptance. The learned model decides where to spend the next unit of search.

A practical architecture can combine exact and heuristic components:

$$\begin{aligned} &\text{exact rational costs} + \text{incumbent bound} + \text{admissible lower bounds} \\ &\quad + \text{learned ranking} + \text{portfolio diversity} + \text{verified bank.} \end{aligned}$$

The exact pieces prune safely; the learned pieces reorder or allocate the remaining work.

6 The theoremhood quotient: proofs as representatives

The maze picture suggests a deliberately coarse equivalence relation on successful routes. Fix a formal system F , hypotheses Γ , target theorem φ , and a theorem maze whose edge labels record the inference applications used along a directed walk. Let $\Pi(P)$ denote the pullback of a finite proof tour P to its corresponding finite sequence of well-formed formulas (WFFs). Let $\mathcal{W}_\varphi$ be the set of finite directed tours from the entrance to a designated φ -exit, and let

$$\mathcal{P}_\varphi = \{P \in \mathcal{W}_\varphi : \Pi(P) \text{ is a verifier-accepted proof of } \varphi \text{ from } \Gamma\}.$$

Here “tour” means a finite directed walk from the entrance to an exit; no closed-walk convention is intended.

Definition 6.1 (Logical proof-tour relation). *For $P, Q \in \mathcal{W}_\varphi$, write*

$$P \sim_\varphi Q \iff \Pi(P) \text{ and } \Pi(Q) \text{ are both proofs of } \varphi \text{ from } \Gamma.$$

Theorem 6.2 (Proof-Tour Equivalence Theorem). *Restricted to the successful proof-tour set $\mathcal{P}_\varphi$, the relation $\sim_\varphi$ is an equivalence relation. Equivalently, for any $P, Q \in \mathcal{P}_\varphi$,*

$$P \sim_\varphi Q \iff \Pi(P) \text{ and } \Pi(Q) \text{ are both proofs of } \varphi \text{ from } \Gamma.$$

If $\mathcal{P}_\varphi \neq \emptyset$, then every two successful tours are equivalent and therefore

$$\mathcal{P}_\varphi / \sim_\varphi = \{[\varphi]\}.$$

Thus the quotient forgets how the theorem was proved and remembers only the verified theoremhood represented by the successful class.

Proof. Every $P \in \mathcal{P}_\varphi$ pulls back by definition to a verified proof of φ from Γ . Hence every pair $P, Q \in \mathcal{P}_\varphi$ satisfies $P \sim_\varphi Q$. The restricted relation is therefore the universal relation on $\mathcal{P}_\varphi$, which is reflexive, symmetric, and transitive. If the set is nonempty, its quotient has exactly one class. $\square$

This is the coarsest logical quotient for a fixed theorem. A short proof, a long proof, a proof discovered by BFS, a proof discovered by DATA, and any other verifier-accepted proof of the same φ are different representatives of the same logical class $[\varphi]$. The route still matters computationally, but not for the bare fact $\Gamma \vdash_F \varphi$.

Corollary 6.3 (Proof Horizon as minimum representative cost). *Let $w : E \rightarrow \mathbb{Q}_{\geq 0}$ be the declared exact rational edge-cost map. Define the class cost by*

$$H_w(\Gamma, \varphi) = \inf_{P \in [\varphi]} w(P).$$

Whenever the presentation guarantees that the infimum is attained (for example, in a finite maze, an effectively proper bounded layer, or the unit-cost proof-length setting),

$$H_w(\Gamma, \varphi) = \min_{P \in [\varphi]} w(P).$$

Thus the weighted Proof Horizon is naturally a property of the entire theoremhood class rather than of any one chosen proof.

In the unit-cost normalization

$$w(e) = 1 \quad \text{for every inference edge } e,$$

we obtain

$$w(P) = |P|$$

and hence, whenever $\Gamma \vdash_F \varphi$,

$$H_1(\Gamma, \varphi) = \min_{P \in [\varphi]} |P|,$$

the shortest proof length. This recovers the unit-weight BFS/Proof-Horizon picture in quotient language: the theorem is the equivalence class, a concrete proof is a representative, and the Horizon is the cost of the cheapest representative.

Caution 6.4 (Logical quotient versus operational compression). The theoremhood quotient is conceptually strong but does not by itself supply a search shortcut. To recognize that an unexplored tour belongs to $[\varphi]$ is already to establish that its pullback proves φ . Operational compression therefore requires finer effectively recognizable quotients or canonicalizations—for example, commuting independent inference steps, verified DAG identities, syntactic normal forms, or other verifier-respecting reductions that preserve the chosen reachability and cost objective. The theoremhood quotient supplies the semantic endpoint against which those finer reductions can be compared.

7 Purpose and scope

The motivating observation is that several strands of the earlier program fit into one diagram:

$$\text{search dynamics} \longrightarrow \text{cumulative verified history} \longrightarrow \text{SIC} \longrightarrow \begin{cases} \text{ATP,} \\ \text{ALD,} \\ \text{AMLD.} \end{cases}$$

A rich implementation such as $\text{DATA}(x)$ is then a special case. The general mathematics should not depend on the names Professor, Counselor, Picard, Dreamer, Horizon, Prover, Refuter, or

Independence. Those modules matter operationally, but the structural results concern any verifier-backed staged search process satisfying the stated hypotheses.

The central computational premise is equally modest. In many theorem-proving or conjecture-settling instances, a valid certificate and hence a terminal state exists mathematically, while available hardware may not reach it. The principal problem is therefore not fixed-point existence but *computational accessibility*: how to shorten the hitting time, reduce the number of states that must be examined, compress repeated structure, and spend resources on branches that are more likely to reach a certificate.

8 General verified transition systems

Definition 8.1 (Verified transition system). *A verified transition system is a tuple*

$$\mathfrak{M} = (X, \mathcal{F}, \pi, V, \mathcal{R})$$

where X is a set of complete instantaneous states, $\mathcal{F} = \{F_j : X \rightarrow X\}_{j \in J}$ is a family of admissible state transformations, π is a controller or scheduling policy selecting transformations, V is an exact verification gate for mathematical deposits, and $\mathcal{R}$ is the space of typed verified records.

A run is a sequence

$$x_0, x_1, x_2, \dots, \quad x_{n+1} = F_{j_n}(x_n),$$

with j_n selected by π or by an allowed nondeterministic/stochastic rule. The state x_n may include queues, heuristic parameters, random seeds, learned representations, resource allocations, untrusted proposals, checkpoints, and verified mathematical memory.

Caution 8.2 (The complete state need not be monotone). Nothing requires $x_n \subseteq x_{n+1}$ or any analogous order relation. A practical searcher may discard a queue, restore a checkpoint, lower a beam width, change a representation, or restart. Monotonicity will be imposed only on the verified-history projection.

9 The cumulative history of verified information

This construction is the bridge from general search dynamics to SICs.

Definition 9.1 (Verified stage output). *Let V_n denote the set of mathematical records newly accepted by the independent verifier during the transition into stage n . Set $V_1 = \emptyset$ by convention, so that stage 1 is exactly the input stage. Depending on the machine, later V_n may include formulas, proved lemmas, proof certificates, refutation certificates, independence or dependence certificates, model witnesses, provenance data, formally checked derived rules, or other typed mathematical objects.*

Definition 9.2 (Cumulative history of verified information). *For input Γ and stage $n \geq 1$, define*

$$C_{\mathfrak{M}}(\Gamma, n) = \Gamma \cup \bigcup_{k=1}^n V_k.$$

In particular, $C_{\mathfrak{M}}(\Gamma, 1) = \Gamma$. This is the cumulative history of verified information through stage n .

The phrase is meant literally. $C_{\mathfrak{M}}(\Gamma, n)$ is not a snapshot of everything the machine currently believes, proposes, scores, or searches. It is the permanent ledger of what has crossed the verification boundary. Search states may be abandoned; verified mathematics is not silently revoked.

Proposition 9.3 (Cumulative-history monotonicity). *For every run and every n ,*

$$C_{\mathfrak{M}}(\Gamma, n) \subseteq C_{\mathfrak{M}}(\Gamma, n+1).$$

Proof. By definition,

$$C_{\mathfrak{M}}(\Gamma, n+1) = C_{\mathfrak{M}}(\Gamma, n) \cup V_{n+1}.$$

□

Remark 9.4 (Projection, not necessarily quotient dynamics). The cumulative history is a monotone observation of a possibly nonmonotone machine. Two complete states can have the same cumulative verified history while differing in queues, resources, or controller state and therefore having different futures. Thus the cumulative history need not be Markov-sufficient. It is a canonical SIC state even when it is not by itself a deterministic dynamical state.

10 The cumulative-history SIC theorem

The SIC framework used in *Depths of a Simulation* consists of a stage-state map C and halting map H with an initial state, monotone accumulation, persistent halting, and freezing after halting [14].

Definition 10.1 (Certificate halting). *Fix a target condition $Z \subseteq \mathcal{R}$ of accepted terminal records. Define*

$$H_{\mathfrak{M}}(\Gamma, n) = 1 \iff C_{\mathfrak{M}}(\Gamma, n) \cap Z \neq \emptyset.$$

After the first such stage, the operational machine may halt. For SIC analysis, extend the halted computation constantly thereafter.

Theorem 10.2 (Cumulative-History SIC Theorem). *Suppose verified records are persistent and the computation is extended constantly after its first accepted terminal certificate. Then*

$$M_{\mathfrak{M}} = (C_{\mathfrak{M}}, H_{\mathfrak{M}})$$

is a semi-ideal computer on the chosen verified-record universe.

Proof. The initial condition is fixed by construction. Monotonicity is the previous proposition. If $H_{\mathfrak{M}}(\Gamma, n) = 1$, then the accepted terminal record remains in every later cumulative history, so halting persists. By the constant extension convention, the cumulative state is frozen after the operational halt. □

11 When the induced SIC is an ATP, ALD, or AMLD

11.1 ATP specialization

If the verified records are formulas and proof certificates in a frozen formal system, and the machine halts on target φ exactly when a verifier-accepted proof of φ is in the cumulative history, then the induced SIC is a target-search ATP once the usual soundness and eventual-success conditions are imposed. The internal search machinery can be arbitrarily elaborate; the SIC sees only the monotone verified ledger.

11.2 ALD specialization

For a target sentence φ over theory G , use typed terminal certificate classes

$$\begin{aligned} P : G \vdash \varphi, \quad R : G \vdash \neg\varphi, \\ I : \text{Ind}_G(\varphi), \quad \perp : \text{certified inconsistency of the base presentation.} \end{aligned}$$

An ALD is then a typed cumulative-history SIC whose halting condition is the appearance of an accepted supported settlement certificate.

11.3 AMLD specialization

At the metalogical level, define

$$\text{Dep}_G(\varphi) := (G \vdash \varphi) \vee (G \vdash \neg\varphi), \quad \text{Ind}_G(\varphi) := \neg \text{Dep}_G(\varphi),$$

and type every meta-certificate by an explicit metatheory. An AMLD is the same SIC-lift principle applied one logical level higher. Timeout, stagnation, and budget exhaustion remain UNKNOWN unless a certificate language explicitly proves otherwise.

12 Halting and absorbing fixed points

Let $T : X \rightarrow X$ be a deterministic macro-update for a complete-state model and let $\mathcal{S} \subseteq X$ be the certified settlement states. Extend terminal states as absorbing:

$$x \in \mathcal{S} \implies T(x) = x.$$

If every nonterminal complete state changes under T , then

$$\text{Fix}(T) = \mathcal{S}.$$

This is the fixed-point settlement pattern proved for DATA 3.0 under an explicit round-counter hypothesis [19].

Proposition 12.1 (Fixed-Point/Halting Correspondence). *Assume $\text{Fix}(T) = \mathcal{S}$, and define $H(\Gamma, n) = 1$ exactly when $x_n \in \mathcal{S}$. Then*

$$H(\Gamma, n) = 1 \iff x_n \in \text{Fix}(T).$$

Caution 12.2 (Existence is not reachability). The identity $\text{Fix}(T) = \mathcal{S}$ characterizes terminal states; it does not imply that a given orbit reaches one. A fixed point can exist while its hitting time exceeds every practical hardware, memory, or wall-clock budget. The search problem is therefore to reduce the cost of reaching a fixed point, not merely to prove that fixed points exist.

13 Control AMLD theory: from static compassing to feedback settlement

The preceding sections separate logical authority from search policy: the verifier alone decides whether a certificate is accepted, while learned or hand-designed components decide where computational effort is spent. The next step is to make that policy itself a mathematical object. A *control AMLD* is an AMLD equipped with a feedback controller whose actions allocate search effort among proof, refutation, independence, representation change, and meta-level reasoning while preserving conservative certificate semantics.

13.1 State, actions, and the control objective

Let X_t denote the complete unsettled search state at control epoch t . It may contain the typed verified bank, the $P/R/I$ frontiers, learned compass features, resource counters, current incumbent certificates, and meta-level information. Let the action u_t encode a resource allocation or search intervention, for example

$$u_t = (r_P, r_R, r_I, r_M, a_{\text{repr}}, a_{\text{bank}}), \quad r_i \geq 0, \quad \sum_i r_i \leq R_t.$$

The controlled transition law may be stochastic,

$$X_{t+1} \sim K(\cdot \mid X_t, u_t),$$

while the settlement set $\mathcal{S}$ remains verifier-defined and absorbing. The controller does not certify theoremhood, refutability, or independence; it chooses computation intended to reach one of the accepted certificate classes sooner.

A basic finite-horizon objective is

$$J_\pi(x; B) = \Pr_x^\pi(\tau_{\mathcal{S}} \leq B),$$

and a cost objective is

$$V_\pi(x) = \mathbb{E}_x^\pi \left[\sum_{t < \tau_{\mathcal{S}}} c(X_t, u_t) \right],$$

with exact rational transaction costs available when desired. On a finite controlled theorem maze, standard Bellman recursion therefore supplies an exact benchmark value function against which a learned compass or scheduler can be compared. This makes control AMLD theory a bridge between the rational theorem-maze layer and the learned navigation layer rather than a replacement for either.

13.2 Abel coordinates and control-Lyapunov coordinates

The functional-equation viewpoint suggests searching for a scalar coordinate A in which successful progress is approximately translation:

$$A(Tx) - A(x) \approx 1.$$

Equivalently, with $V = -A$ up to an additive constant, useful actions should make a settlement-distance surrogate decrease. The relevant empirical quantity is not merely whether A correlates with proof depth, but whether the *executed controlled increments*

$$\Delta A_t = A(X_{t+1}) - A(X_t)$$

have positive drift and tolerable variance on held-out trajectories. A natural diagnostic is the Abel residual

$$\epsilon_A = \mathbb{E}[A(X_{t+1}) - A(X_t) - 1].$$

This is stricter than a static rank metric: a model can rank proof-DAG nodes well yet choose an off-DAG successor and immediately destroy the trajectory.

Theorem 13.1 (Abel-drift settlement bound). *Let $D(X_t) \geq 0$ before settlement and suppose that, for some $\mu > 0$,*

$$\mathbb{E}[D(X_{t+1}) - D(X_t) \mid \mathcal{F}_t] \leq -\mu$$

whenever $X_t \notin \mathcal{S}$. Under the standard optional-stopping integrability conditions, if τ is first settlement time, then

$$\mathbb{E}[\tau] \leq \frac{D(X_0)}{\mu}.$$

Proof. The stopped process $M_t = D(X_{t \wedge \tau}) + \mu(t \wedge \tau)$ is a supermartingale. Hence

$$\mathbb{E}[D(X_{t \wedge \tau})] + \mu \mathbb{E}[t \wedge \tau] \leq D(X_0).$$

Discard the nonnegative first term and pass to the limit under the stated integrability hypotheses. $\square$

Theorem 13.2 (Finite-budget variance penalty). *Assume progress increments have mean $\mu > 0$ and variance σ^2 , with independence or martingale-difference hypotheses sufficient for additive variance. If cumulative progress D is required and $B\mu > D$, then Cantelli's inequality gives*

$$\Pr\left(\sum_{t=1}^B \Delta A_t < D\right) \leq \frac{B\sigma^2}{B\sigma^2 + (B\mu - D)^2}.$$

Thus a control signature should include variance as well as drift. A policy with slightly smaller mean progress but much smaller variance may be preferable under a hard proof budget.

Theorem 13.3 (Robust Lyapunov control under compass error). *Suppose a true control-Lyapunov quantity $V \geq 0$ admits an ideal action with expected one-step decrease at least $\alpha > 0$. If the learned action loses at most ϵ expected decrease relative to the ideal action at each unsettled state, then*

$$\mathbb{E}[V(X_{t+1}) - V(X_t) \mid X_t] \leq -(\alpha - \epsilon).$$

If $\epsilon < \alpha$, the same drift argument yields

$$\mathbb{E}[\tau] \leq \frac{V(X_0)}{\alpha - \epsilon}.$$

This result gives a useful interpretation of compass error: the relevant question is whether approximation regret is smaller than the available control margin, not whether the learned distance is numerically exact.

13.3 P/R/I control and resource allocation

Equal-turn scheduling is a strong neutral baseline because it prevents starvation. Once agent-specific short-horizon response curves can be estimated, however, a feedback controller can ask a more precise question: where does the next unit of computation have the highest marginal settlement value?

Theorem 13.4 (Concave resource water filling). *Suppose m AMLD agents receive resources $r_i \geq 0$ with $\sum_i r_i = R$, and agent i contributes an increasing differentiable concave short-horizon settlement hazard $h_i(r_i)$. Any maximizer of*

$$H(r) = \sum_i h_i(r_i)$$

satisfies the KKT conditions

$$h'_i(r_i) = \lambda$$

for active agents, while inactive agents satisfy $h'_i(0) \leq \lambda$.

The theorem does not establish that real $P/R/I$ response curves are additive or concave. It states what optimal allocation looks like if those measurable assumptions are approximately true over a control epoch. This yields a direct experiment: estimate h_P, h_R, h_I on sealed ALD/NOTALD tasks, compare water filling with equal-turn scheduling, and test whether a low-dimensional switching surface predicts reallocations.

13.4 Five deliberately open control-AMLD conjectures

The following are research targets rather than established properties of theorem search.

- C1. Low-dimensional performance signature.** A small vector of quantities such as global discrimination, distance-order quality, local tunneling quality, drift, variance, and Abel residual predicts held-out settlement probability and effort across broad theorem families.
- C2. Emergent Abel coordinate.** With sufficient true proof-DAG training, a scalar A emerges for which successful controlled transitions have an approximately stationary residual $A(Tx) - A(x) - c$ on unseen theorem families.
- C3. Control-Lyapunov equivalence.** On a broad finitely branching class, highly reliable settlement guidance is essentially equivalent to the existence of a learnable scalar or low-dimensional control-Lyapunov representation.
- C4. P/R/I switching surfaces.** With a shared verified bank and online hazard estimates, near-optimal resource control can be represented by relatively simple switching surfaces rather than long-memory chaotic alternation.
- C5. Fast-global/slow-local learning.** Compass learning has at least two distinct scales: global proof-geometry discrimination matures relatively early, while local direct-parent tunneling requires substantially more true DAG data.

14 Our settlement-compass experiments

The experimental program in the public ATP repository evolved in stages. Experiments 001 and 002 established the navigation-diagnostic machinery and small-DAG comparisons. Experiment 003 introduced shell-size comparisons. Those early studies were useful engineering and exploratory work, but changing training histories or candidate construction could confound a causal interpretation of training-set size. Experiment 004 was therefore redesigned as the primary clean learning-curve experiment. Experiment 005 then began the more demanding transition from static DAG ranking to executed control trajectories and Abel/Bellman diagnostics.

The public result directory is

https://github.com/btenneson/ATP/tree/main/optimizations/settlement_compass/results.

14.1 Experiment 004: frozen nested shells and the MAX “stupid case”

One eligible proof-DAG universe was split once into a sealed twenty-target test set and a training pool. Nested shells were taken from one frozen ordering,

$$S_{10} \subset S_{20} \subset S_{40} \subset S_{80} \subset S_{160} \subset S_{\max},$$

with every shell trained from scratch under the same feature coordinates, negative samples, test candidate pools, seeds, and model class. The deliberately oversized MAX condition used all 3,612 eligible remaining true training roots. This was designed to reveal whether additional true DAG data still mattered after the ordinary shells appeared to saturate.

Training roots	AUC	Distance ρ	MAE	P@10	Median direct-parent rank
10	0.8353	0.2398	2.5042	0.450	569
20	0.8650	0.2784	2.4310	0.415	237.5
40	0.8761	0.3149	2.5080	0.490	219
80	0.8876	0.3318	2.7019	0.570	251
160	0.8880	0.3427	2.8086	0.555	199
3612 (MAX)	0.8824	0.3483	2.3993	0.670	57

Table 3: Experiment 004 aggregate results on one sealed twenty-target set. The experiment is a retrospective proof-DAG navigation diagnostic, not an end-to-end ATP race.

The global classifier essentially stops improving in AUC by the 80–160 region, whereas local navigation remains sensitive to the very large corpus. At MAX, precision@10 reaches 0.670, median first direct-parent rank falls to 57, and the compass beats random direct-parent ordering on 17/20 targets, versus 12/20 at the 160 shell. The MAX classifier used roughly 1.98 million classifier examples and 1.37 million regression examples.

14.2 Illustrative learning-curve fits

With only six aggregate training conditions, curve fitting must remain descriptive. The purpose is to estimate plausible scales and design the next shells, not to declare a universal learning law. A simple saturation model is

$$y(n) = y_{\infty} - be^{-n/\tau}.$$

Least-squares fits to the current aggregate values give:

Metric and fitting range	y_{∞}	τ	R^2	Interpretation
AUC, shells 10–160	0.8868	14.3	0.982	Very fast global-discrimination saturation; MAX AUC 0.8824 is slightly below the fitted plateau.
Distance Spearman, shells 10–160	0.3397	21.8	0.996	Global distance ordering also matures quickly; MAX 0.3483 modestly exceeds the five-shell extrapolation.
Precision@10, all six conditions	0.6643	137.6	0.893	A substantially slower local/top- k scale is compatible with the data.

Table 4: Illustrative one-timescale fits. The fitted τ values are model parameters, not proven sample-complexity constants.

Under this model the P@10 timescale is about 9.6 times the AUC timescale. That is quantitatively consistent with the fast-global/slow-local hypothesis, but the interval $160 \rightarrow 3612$ is too sparse to identify a genuine local exponential scale. A different low-parameter model also fits the local statistic reasonably well: on all six points, a log–log regression gives approximately

$$\text{MedRank}_{\text{parent}}(n) \approx 928 n^{-0.335},$$

with $R^2 \approx 0.889$ in log space. Because both saturation and power-law descriptions remain plausible at this resolution, the correct conclusion is experimental: add intermediate shells, not choose a favorite curve prematurely.

The highest-value interpolation points are therefore roughly

$$n \in \{320, 640, 1280, 2560\},$$

with the original twenty test targets and every other frozen experimental choice unchanged. Model comparison should be done on predictive error, preferably by withholding one or more shell conditions or, better, by repeating the entire shell protocol on unrelated theorem families.

14.3 Experiment 005: the controller is harder than the compass

Experiment 005 extends the program to “interpolation + retrospective executed DAG control trajectories” and explicitly remains *not* a live ATP race. The trajectory diagnostics begin measuring quantities required by the control theorems: learned and true drift, increment variance, Abel residual, Bellman-value error, hitting time, and trajectory settlement.

One currently recorded aggregate at the 160-DAG shell is particularly instructive. Across 100 retrospective trajectories, the learned progress coordinate had estimated drift

$$\hat{\mu} \approx 0.0526,$$

learned increment variance about 0.659, mean learned Abel residual about 1.016, and Bellman-value MAE about 2.156 on executed states, yet the recorded trajectory success rate was

$$0/100.$$

This is not a contradiction of the drift theorem: its hypotheses require a suitable uniform conditional decrease toward the actual settlement set, not merely a positive aggregate statistic on a mixture of transitions. The trajectories frequently terminate by selecting distractor/off-DAG states. The result is therefore scientifically useful: it shows exactly why control-AMLD evaluation must measure closed-loop trajectories rather than infer control quality from AUC, Spearman correlation, or static top- k ranking alone.

14.4 What the experiments currently support

The present evidence is strongest for three claims of limited scope:

1. more true proof-DAG training measurably improves several held-out navigation statistics, with diminishing returns in global AUC;
2. global discrimination and local tunneling need not saturate at the same training size;
3. a static compass can look useful while a naive closed-loop controller still fails, so successor selection, off-DAG robustness, and control stability are independent research problems.

The experiments do *not* yet establish an end-to-end AMLD speedup, an optimal P/R/I resource policy, an Abel coordinate with stationary residual, or a universal two-timescale law.

14.5 Next control experiments

The next experiments should be deliberately causal and controller-facing:

- (i) fill the $160 \rightarrow 3612$ interval with frozen intermediate shells and refit one-, two-, and simple power-law learning curves;
- (ii) log complete executed trajectories so that μ , σ^2 , ϵ_A , failure mode, and hitting time are measured per theorem rather than only in aggregate;
- (iii) on finite proof DAGs, compute the exact Bellman value function and compare the learned controller’s action regret with the true one-step optimal action;
- (iv) run equal-turn versus hazard-water-filling $P/R/I$ schedules on sealed NOTALD controls before interpreting any gain on genuine independence problems;
- (v) include deliberate detour mazes where every successful route must temporarily increase a naive distance, testing whether scalar Lyapunov control is sufficient or whether vector/history-valued state is required;
- (vi) retain an exhaustive/fair background search so that aggressive control changes order and resource allocation without silently deleting certificate-complete coverage.

The control-AMLD program therefore turns a broad slogan—“make the search more intelligent”—into measurable quantities. The central empirical object is no longer just a classifier score; it is the distribution of verifier-respecting settlement trajectories under feedback.

15 Prior synthetic benchmark evidence, with disclaimers

The technical companion to the Proof Horizon reported an $L^* = 100$ synthetic dry tournament on 20 independently seeded finite “oceans.” Each ocean had an externally established shortest successful route of exactly 100 edges. General searchers were not told $L^* = 100$ and were not given the hidden shortest route. The stopping rule was first verifier-accepted proof [17].

System	Finished	Optimum first	Median L	Median E	Median time (ms)
Depths-F specialized control	20/20	20/20	100	100	0.002
BFS	20/20	20/20	100	10928.5	3.013
DFS	20/20	4/20	115	1747.5	0.487
Beam-64	19/20	16/20	100	5546.5	2.664
Data-ATP Hilbert-only ablation	20/20	6/20	115	1317.5	7.115
Data-ATP full dry stand-in	20/20	11/20	100	375	30.747
DATA 2.0.1 fast wing	20/20	7/20	115	121.5	28.570
DATA 2.0.1 Horizon certifier	20/20	20/20 certified	100	10928.5	3.037

Table 5: Previously reported synthetic dry results at $L^* = 100$; reproduced here only as architectural evidence and benchmark context, not as a production-performance claim.

Required disclaimer. These numbers are *not* production benchmarks of the full Data-ATP or DATA repositories, do not establish superiority on natural mathematics, and should not be read as a universal speedup theorem. They are executable stand-ins designed to separate intrinsic path length from navigation overhead and to test measurement, failure reporting, and architecture. In that dry setting, the fuller Data-ATP stand-in used far fewer median expansions than BFS while the separate Horizon certifier recovered/certified the known optimum at exhaustive-search cost. The result motivates, but does not validate, the larger search-reduction program [17].

16 A proposed rational-weight theorem-maze benchmark

The next benchmark should test the graph-theoretic claim directly and should use exact rational weights rather than floating-point surrogates.

Definition 16.1 (Rational Ocean benchmark family). *A benchmark instance is a finite or effectively bounded directed theorem maze with:*

1. *a planted or independently certified minimum settlement cost $L_w^* \in \mathbb{Q}_{\geq 0}$;*
2. *exact edge weights a/b stored in reduced rational form;*
3. *a frozen verifier and certificate language;*
4. *adjustable width, dead-end density, alternative successful routes, repeated substructures, and misleading locality;*
5. *optional typed exits for P , R , I , and $\perp$;*
6. *a hidden shortest route unavailable to general searchers.*

The benchmark should compare at least a rational Dijkstra baseline, rational A* with frozen admissible heuristics, BFS on unit-weight controls, beam/best-first variants, portfolio search, shared-bank ablations, and the relevant ATP/ALD/AMLD architecture. Every addition must preserve exact certificate checking.

Primary measurements should include verified settlement rate and type, first-settlement cost, certified minimum settlement cost where attempted, node/edge expansions, wall-clock time, peak memory, verifier time, bank deposits and retrievals, restarts, and the gap

$$\Delta = \text{Cost}(\text{first verified settlement}) - L_w^*.$$

For minimum-cost experiments, report separately the cost of discovery and the cost of certification.

Benchmark disclaimer. A rational-weight synthetic maze is a controlled model of proof-search geometry, not a claim that real theorem libraries have the same distribution. Success is meaningful only if ranking survives held-out changes in width, decoys, weight denominators, representation, and eventually frozen natural formal-mathematics benchmarks.

17 Reflective ATPs: proving things about the prover

Once the cumulative-history SIC satisfies ATP conditions, the induced ATP can itself be given a formal target language containing an effective code of a frozen machine M .

Let $M^\#$ be a coded or tagged copy of the machine, in the spirit of the tagging constructions in the author's *Depths* line [14]. The meta-ATP can then search for derivations of statements such as

“every accepted P -certificate of $M^\#$ verifies,”

“the terminal states of $M^\#$ are absorbing,”

“this scheduler is fair on this finite benchmark,”

or bounded search facts such as

“no certificate of declared cost $< B$ occurs in this effectively finite layer.”

If such a theorem is independently verified, it may be deposited in a typed meta-level bank and used by a controller as mathematical information about the search process.

Theorem 17.1 (Meta-ATP construction, schematic form). *Suppose the cumulative-history lift of M is an effective ATP A_M , and the background formal theory effectively represents a frozen code $M^\#$ together with the relevant syntactic verification predicates. Then A_M can search for every derivable encoded property of $M^\#$ expressible in that theory. Any accepted result is a theorem of the background formal system, not an oracle statement about semantic truth.*

Caution 17.2 (No unrestricted self-certification). This construction does not evade the classical incompleteness, undecidability, or undefinability barriers associated with Goedel, Church, and Tarski [21, 22, 23]. It is most credible for bounded, syntactic, relative, conditional, and verifier-checkable properties. A machine proving a coded local fact about its search is not the same thing as a machine proving its unrestricted global soundness.

18 DATA as a distinguished case

DATA 3.0 is a rich instance of the general framework. Its complete state includes object theory, target, metatheory, typed verified bank, multiple search frontiers, untrusted proposals, history, learned parameters, resource allocation, candidate/accepted certificates, settlement status, and a monotone round counter [19]. Its active subsystems are modeled as transformations whose compositions generate a transformation semigroup.

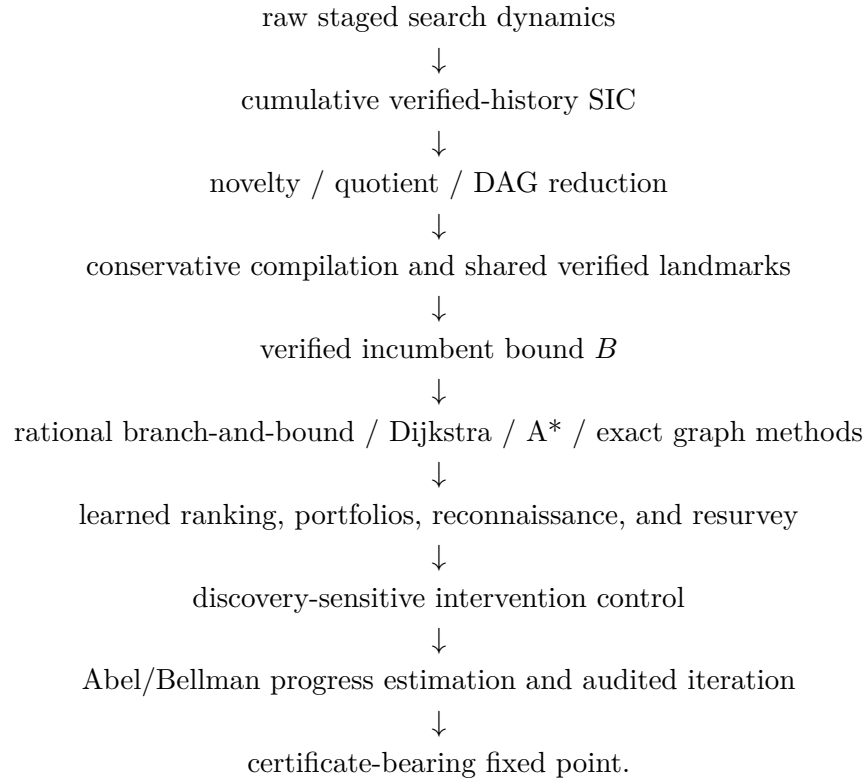
The general theory above interprets DATA in four simultaneous ways:

1. the instantaneous DATA process is a controlled transition system or IFS-like semigroup action;
2. its cumulative verified history is a SIC;
3. the P route specializes to an ATP, while the combined $P/R/I$ routes form an ALD and the nested dependence/independence route forms an AMLD;
4. the resulting theorem/settlement graph is a rationally weighted maze once exact rational transaction costs are declared.

The DATA-specific intelligence layer can therefore be understood as a collection of maze-navigation policies. Professor scores frontiers; Counselor challenges representations; Picard supplies feedback; Creativity and Dreamer generate candidate moves; Horizon certifies bounded regions; the verified bank adds reusable macro structure. None of these changes the verifier's semantics.

19 A search funnel for computational accessibility

The synthesis of the Horizon and Depths programs suggests the following funnel:



The strongest claim is not that the fixed point becomes easy. It is that each justified transformation can remove a class of states or routes that do not need to be searched for the declared objective, while the discovery layer can learn which surviving transformation family is worth trying next and the Abel layer can measure graded motion when the binary endpoint is too coarse.

20 Research conjectures and implementation questions

Conjecture 20.1 (Verified-history compression). *On broad formal-mathematics families, searching a canonicalized graph of cumulative verified histories requires asymptotically or practically fewer expansions than searching raw transient histories under matched correctness constraints.*

Conjecture 20.2 (Compiled-landmark acceleration). *A verified shared bank plus conservative macro compilation reduces rational weighted hitting time on families with repeated substructure, after retrieval and verification overhead are charged exactly.*

Conjecture 20.3 (Admissible learned lower bounds). *For some nontrivial theorem families, a learned function can be calibrated into a valid rational lower bound $h(v)$ often enough to improve branch-and-bound or A^* search while preserving exact minimum-cost guarantees.*

Conjecture 20.4 (Reflective search improvement). *A meta-ATP can prove bounded or conditional facts about a frozen source ATP/ALD/AMLD that, when deposited into the verified meta-bank, measurably reduce later settlement cost without altering the verifier or certificate semantics.*

Conjecture 20.5 (Discovery transfer). *There exist formal-mathematics families and verifier-respecting state summaries for which intervention outcomes on earlier solved rungs predict, above matched baselines, which class of intervention increases the probability of a verified novel useful transition on a later hidden rung. The intended claim is empirical and forward-chaining: the policy is frozen before the later rung is exposed.*

Conjecture 20.6 (Transferable approximate Abel coordinate). *There exist nontrivial theorem families for which an Abel/Bellman coordinate learned only from earlier audited rungs has positive out-of-sample discovery drift, bounded residual, and sufficient action-gap accuracy to predict useful intervention choices on later sealed rungs better than the ordinary-search baseline.*

Conjecture 20.7 (Discovery-control escalation). *If a discovery controller is genuinely learning mechanism rather than one-family syntax, then its prospectively measured intervention advantage should survive at least one sealed cross-family replication after the feature map, action grammar, and controller have been frozen.*

Key implementation questions include whether cumulative-history canonicalization is cheap enough to pay for itself; how to define rational edge costs that remain meaningful across machines; how much of proof-state equivalence can be checked exactly; whether macro compilation should affect search cost, native proof cost, or both; which meta-theorems are strong enough to improve scheduling rather than merely restating architecture; whether a low-dimensional Abel coordinate exists on discovery trajectories; when scalar drift must be replaced by vector or history-valued state; and whether a system can learn, from earlier settled rungs, when a qualitatively different mathematical mechanism is becoming more valuable than continued local search.

21 Discovery

The preceding sections ask how a theorem maze can be compressed, bounded, navigated, and controlled. A different question is what it means for such a search to *discover* something. For an automated system, this word should not be left as a synonym for “the prover happened to finish.” A useful formalization should distinguish a routine certificate from a genuinely new state transition, should be measurable on already solved problems, and should support a controller without permitting failure, timeout, or surprise to masquerade as mathematical knowledge.

This section therefore treats discovery as a verifier-backed event with graded precursors and consequences. The endpoint can be binary once the experiment declares its equivalence relation and usefulness criterion; the approach to that endpoint need not be binary. Discovery readiness, finite-horizon discovery hazard, mechanism probability, discovery depth, and intervention gain can all be measured continuously.

21.1 Weak and operational discovery

Let X_t be the complete operational state at stage t , including the verified bank, active goals, representation, frontier, controller state, resources, and proposal queues required to make the experiment reproducible. Let B_t denote the cumulative verified bank, and let q be a newly verifier-accepted mathematical record produced from X_t .

Fix a verifier-respecting equivalence relation $\sim$ on records. The relation is part of the experimental declaration. It may identify alpha-renamings, canonicalized proof states, or other transformations whose preservation theorem has actually been established; it must not silently identify states merely because a heuristic says they look similar.

Definition 21.1 (Weak discovery). *A verified record q is a weak discovery at stage t when*

$$V(q) = 1 \quad \text{and} \quad [q]_{\sim} \notin B_t / \sim,$$

where V is the independent verifier gate.

Weak discovery captures certified novelty. It deliberately does not require that every new lemma be important. In particular, the quotient prevents an ATP from claiming unlimited substantive discovery merely by producing syntactic variants when the declared equivalence already identifies them.

To formalize usefulness, fix a target distribution μ , a controller π , and a finite resource budget R . Let $D_{\pi}(C \mid X)$ be the settlement cost of target C from state X , with value $+\infty$ if settlement does not occur. Define the capped finite-budget cost

$$D_{\pi,R}(C \mid X) := \min\{D_{\pi}(C \mid X), R + 1\}.$$

Thus a failure to settle inside budget receives the finite penalty $R + 1$. For a verified record q , define

$$U_R(q \mid X_t) = \mathbb{E}_{C \sim \mu} [D_{\pi,R}(C \mid X_t) - D_{\pi,R}(C \mid X_t \oplus q)].$$

Here $X_t \oplus q$ means that q is made available through the declared verified-bank interface, with retrieval and verification overhead charged according to the benchmark protocol.

Definition 21.2 (Operational discovery). *A weak discovery q is an operational discovery at budget R if*

$$U_R(q \mid X_t) > 0,$$

or if it satisfies another prospectively declared usefulness test, such as a strict increase in the verified reachable set within budget R .

The adjective “operational” is essential. This definition does not claim to measure timeless mathematical significance. It measures whether a certified novelty changes what the declared machine can do under a declared workload and budget. Since usefulness may require later evaluation, the operational label can be assigned retrospectively even though the underlying weak-discovery event is recorded immediately.

21.2 Binary event, continuous precursors

For a fixed experiment define

$$Y_t = \begin{cases} 1, & \text{if an operational discovery is credited to stage } t, \\ 0, & \text{otherwise.} \end{cases}$$

Thus the endpoint is binary. The state leading toward that event can nevertheless carry graded information. A practical discovery-readiness vector may include

$$r_t = (N_t, G_t, R_t, L_t, K_t, X_t^{\times}),$$

where, after normalization to a declared scale,

- N_t measures novelty of generated proof structures modulo the chosen quotient;

- G_t measures change in residual-goal signatures rather than repeated return to the same residual state;
- R_t measures representation movement, such as basis, quotient, dual, complement, decomposition, or complexification moves;
- L_t measures transfer of newly verified lemmas into later successful candidates;
- K_t measures compression, for example replacement of recurrent subpaths by one verified reusable construction;
- $X_t^\times$ measures cross-voice transfer, where one coherent proof approach supplies a lemma or representation useful to another.

These coordinates are proposed observables, not axioms of discovery. Their value is determined by predictive and interventional tests on held-out rungs.

21.3 Discovery hazard and discovery depth

Let τ_{disc} be the first stage at which an operational discovery occurs. For controller π and remaining budget R , define the finite-horizon discovery hazard

$$h_\pi(X_t; R) := \Pr_\pi(\tau_{\text{disc}} - t \leq R \mid X_t).$$

The quantity h_π answers a more useful question than a bare retrospective label: conditional on the present state, how likely is the system to produce a discovery within the next declared amount of search?

For a particular discovery class d , define its machine-relative discovery depth by

$$D_{\text{disc}, \pi}(d \mid X_0) := \inf\{\text{Cost}(\gamma) : \gamma \text{ is a verifier-respecting trajectory from } X_0 \text{ to } d\}.$$

This is the discovery analogue of settlement depth or a Proof Horizon. It is explicitly controller- and metric-relative. A mathematically simple idea may have large discovery depth for a machine whose representation makes the relevant transformation nearly inaccessible.

One may also attach a mechanism label

$$M \in \{\text{lemma, quotient, decomposition, duality, representation change, generalization, composition, } \dots\}$$

and estimate both $\Pr(Y = 1 \mid X_t)$ and $\Pr(M = m \mid X_t, Y = 1)$. The first estimates whether discovery is approaching; the second estimates which kind of discovery historically resolved comparable states.

21.4 Discovery gain as a control quantity

Suppose a is an intervention—for example ordinary local proof extension, residual-goal repair, motif development, quotienting, representation modulation, or a counterpoint-style multi-voice exchange. Let a_0 be the declared baseline action. Define the finite-budget discovery gain

$$\text{DG}_R(a, X) = \Pr(\tau_{\text{disc}} \leq R \mid \text{do}(a), X) - \Pr(\tau_{\text{disc}} \leq R \mid \text{do}(a_0), X).$$

Caution 21.3 (Causal language). The do notation is justified only when the experiment actually identifies an intervention, for example by randomized action assignment, matched replay with frozen budgets and seeds, or another design that blocks the relevant confounding. Otherwise the corresponding quantity is an association and should be labeled as such.

A discovery controller can then be posed as

$$a_t \in \arg \max_a \widehat{\text{DG}}_R(a, X_t),$$

subject to certificate-completeness safeguards and any fair-search background process required by the formal setting. This removes the need for a fixed exploration probability ε . Exploration is triggered because a particular intervention has learned positive expected discovery gain in the current state, not because a global coin happens to come up “explore.”

21.5 Retrospective controls on solved rungs

Previously solved theorem rungs provide an unusually clean laboratory because the existence of a certificate is known while the historical proof can be hidden from the experimental search. For each solved rung k , construct a sealed replay in which the system receives only the axioms, verified lower-rung bank, target statement, declared representation interface, and the same verifier used in the live program. Historical candidate bodies, historical ranking traces, and the target’s canonical proof are excluded.

Under a matched verifier-call budget compare intervention families such as

- A : ordinary local search,
- B : motif development,
- C : residual-goal recursive repair,
- D : representation modulation,
- E : counterpoint-style multi-voice search,
- F : a controller selecting among A – E .

Record at least settlement, weak and operational discovery events, verifier calls, wall-clock cost, residual-goal signature changes, representation changes, mechanism labels, lemma transfer, and whether the discovered object changes later held-out settlement cost. Because diagnostic tactics can emit partial suggestions without proving their residual goals, every candidate must still pass the strict certificate gate; diagnostic output may generate descendants but may not certify them.

The crucial control is to hold everything except the intervention fixed. A stronger design randomizes intervention assignment across repeated seeds or matched target instances. A weaker design that changes budgets, candidate banks, and training history simultaneously can estimate performance but cannot cleanly identify discovery gain.

21.6 Forward-chaining rung by rung

Retrospective replay becomes scientifically stronger when converted into a forward-chaining protocol. Let $\mathcal{D}_k$ contain only discovery trajectories from rungs settled before rung k . Train the discovery model on $\mathcal{D}_k$, freeze it, and only then expose the sealed target at rung k . After the rung is settled and its trajectory audited, update

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \{\text{audited trajectory for rung } k\}.$$

No information from rung k may influence the model used to predict rung k . Each rung is temporarily treated as the current frontier. The system can then be evaluated on whether it predicts which intervention family will help, whether its hazard is calibrated, and whether its selected policy lowers discovery depth relative to the frozen baseline.

This protocol can be applied to a current frontier without claiming that earlier mathematics determines later mathematics. The legitimate inference is narrower: if earlier hidden-rung experiments show that certain observable stagnation states are preferentially escaped by representation changes, then a frozen controller may allocate more current resources to representation-changing interventions when the same measured state appears again.

21.7 Motifs, structured variation, and the music analogy

The control language gives a precise place for the structured-music analogy. Let $\mathcal{M}$ be a space of proof motifs: typed proof skeletons, useful lemma configurations, residual-goal patterns, or representations. Let

$$\Gamma = \langle T_{\text{vary}}, T_{\text{transpose}}, T_{\text{invert}}, T_{\text{modulate}}, T_{\text{compose}}, \dots \rangle$$

be a semigroup generated by motif transformations. A search trajectory explores an orbit

$$m, \quad T_1 m, \quad T_2 T_1 m, \quad \dots$$

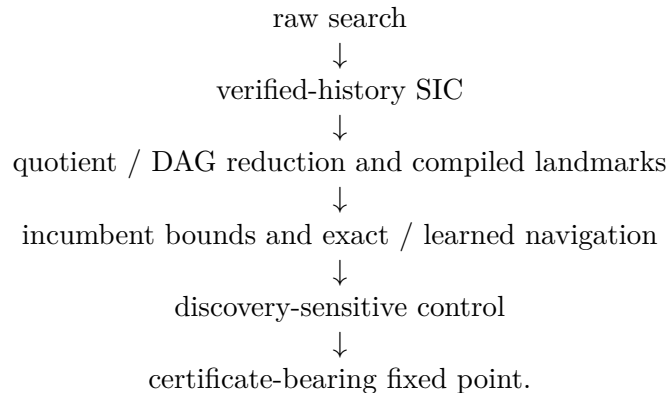
Most orbit points are not discoveries. A point becomes a weak discovery only after independent verification and quotient novelty; it becomes an operational discovery only after the declared usefulness test is met.

This is closer to musical development than to injecting “structured noise.” A motif can be varied, transposed, inverted, combined with another voice, or moved into a new representational key while retaining relations to its source. In proof search, analogous moves include dualization, quotienting, basis change, decomposition, generalization, complexification, or changing from a pointwise to a coordinate representation. Some such transformations need only generate proposals; they gain mathematical authority only when their resulting records survive the verifier. The music analogy supplies candidate transformation families. It is not itself evidence that those transformations improve theorem proving. That question is delegated to the controlled discovery-gain experiment.

21.8 Relation to SICs, ALDs, AMLDs, and fixed points

Discovery fits the earlier SIC framework without changing its semantics. A weak discovery is first of all a verifier-accepted record entering the cumulative history. The SIC state therefore remains monotone. Operational usefulness is an additional measured annotation; it is not needed to preserve verifier soundness or cumulative-history monotonicity. An ATP may discover a new lemma on its path to a proof; an ALD may discover a refutation-supporting or independence-supporting lemma; an AMLD may discover a reusable metatheorem about dependence or independence. None of these labels may be assigned from timeout or failed search.

The search funnel is correspondingly extended:



Discovery-sensitive control does not replace exact search reduction. It acts after the verifier, quotient, and cost semantics have been fixed, using earlier audited trajectories to decide which surviving mathematical transformation is worth trying next.

21.9 What would count as evidence

The discovery formalization should be judged by out-of-sample prediction and controlled intervention, not by whether its terminology sounds plausible. The strongest near-term evidence would be a sequence of sealed rung experiments in which:

1. the discovery model is trained only on earlier rungs and frozen;
2. its hazard estimates are calibrated on held-out trajectories;
3. its chosen mechanism predicts the intervention class that actually produces more verifier-backed discovery under equal budgets;
4. the resulting discoveries measurably reduce later settlement cost or enlarge verified reach; and
5. the advantage survives new theorem families, representations, and random seeds.

A negative result is equally informative. If discovery features fail to predict intervention benefit, the controller should revert to the stronger exact methods already developed in the maze-first program rather than preserve a decorative extra layer.

22 The Abel equation and approximate discovery coordinates

Section 13 introduced Abel coordinates as a possible progress variable for settlement control, while Section 21 separated a binary verifier-backed discovery event from the graded state that precedes it. The two ideas fit naturally together. The classical Abel equation

$$A(Tx) = A(x) + 1$$

conjugates iteration of a transformation T to translation by one unit. In the theory of iterative functional equations this is one of the standard linearizing equations associated with iteration and conjugacy [20]. For theorem search, however, the relevant object is usually not one deterministic map but a controlled, possibly stochastic transition system. The aim is therefore not to assume that an exact global Abel function exists. It is to ask whether a scalar coordinate can be learned on audited trajectories so that motion toward discovery becomes approximately translational and therefore measurable.

22.1 The discovery set and the pre-terminal Abel law

Let $\mathcal{D} \subseteq X$ be the declared discovery set: states in which the current experiment has credited a strict verifier-backed discovery. The set is absorbing for measurement purposes. Thus after $X_t \in \mathcal{D}$ the experiment may freeze the trajectory, and no Abel translation law is required beyond the hitting time

$$\tau_{\mathcal{D}} = \inf\{t : X_t \in \mathcal{D}\}.$$

This restriction matters because an absorbing state has zero increment, whereas the Abel equation describes pre-terminal progress.

Suppose a finite action set $\mathcal{A}$ controls a transition kernel

$$X_{t+1} \sim K(\cdot \mid X_t, a_t),$$

and each action carries an exact nonnegative rational cost

$$c(x, a) \in \mathbb{Q}_{\geq 0}.$$

The unit-cost case is $c \equiv 1$, one unit per control epoch or verifier call.

Define the optimal expected remaining discovery cost

$$V^*(x) = \inf_{\pi} \mathbb{E}_x^{\pi} \left[\sum_{t < \tau_D} c(X_t, a_t) \right], \quad V^*(z) = 0 \quad (z \in \mathcal{D}),$$

whenever this quantity is finite and the usual Bellman principle is valid.

Proposition 22.1 (Bellman-Abel correspondence for discovery). *Assume V^* satisfies the Bellman equation*

$$V^*(x) = \min_{a \in \mathcal{A}} \{c(x, a) + \mathbb{E}[V^*(X_{t+1}) \mid X_t = x, a_t = a]\}$$

for every $x \notin \mathcal{D}$. Set

$$A^*(x) = -V^*(x).$$

Then

$$\max_{a \in \mathcal{A}} \{\mathbb{E}[A^*(X_{t+1}) - A^*(x) \mid x, a] - c(x, a)\} = 0.$$

For every Bellman-optimal action $a^*(x)$,

$$\mathbb{E}[A^*(X_{t+1}) - A^*(x) \mid x, a^*(x)] = c(x, a^*(x)).$$

In the unit-cost case,

$$\max_a \mathbb{E}[A^*(X_{t+1}) - A^*(x) \mid x, a] = 1.$$

Proof. Multiply the Bellman equation by -1 and use $A^* = -V^*$. This gives

$$A^*(x) = \max_a \{\mathbb{E}[A^*(X_{t+1}) \mid x, a] - c(x, a)\}.$$

Subtracting $A^*(x)$ yields the first identity. Equality for an optimal action is immediate, and $c \equiv 1$ gives the final statement. $\square$

This proposition gives a precise sense in which the Abel equation can become involved with discovery. Bellman distance says how much expected discovery cost remains; the negative value A^* is a coordinate in which an optimal transition advances by exactly the cost spent. With unit transaction cost, optimal expected motion is translation by one.

22.2 Abel drift, Bellman-Abel residual, and action scores

An exact A^* will rarely be known on a large implicit theorem maze. Let $\hat{A}$ be an approximation learned from earlier audited rungs. Define the conditional Abel drift of action a by

$$\hat{\mu}_a(x) := \mathbb{E}[\hat{A}(X_{t+1}) - \hat{A}(x) \mid x, a].$$

For unequal action costs, the cost-adjusted score is

$$\hat{s}_a(x) := \hat{\mu}_a(x) - c(x, a).$$

Under the exact Bellman-Abel coordinate, the largest score is zero; optimal actions attain zero and suboptimal actions have nonpositive score. Thus a controller can use

$$\hat{a}(x) \in \arg \max_a \hat{s}_a(x)$$

as a local approximation to the Bellman-optimal discovery action. Under unit costs this is equivalent to maximizing predicted Abel drift.

A statewise Bellman-Abel residual is

$$\rho_{BA}(x) = \hat{A}(x) - \max_a \left\{ \mathbb{E}[\hat{A}(X_{t+1}) \mid x, a] - c(x, a) \right\}.$$

A trajectory residual for a selected action is

$$\epsilon_{A,t} = \hat{A}(X_{t+1}) - \hat{A}(X_t) - c(X_t, a_t).$$

The two residuals answer different questions. ρ_{BA} asks whether the fitted coordinate is approximately Bellman-consistent at the state. $\epsilon_{A,t}$ asks how a particular executed transition departed from the ideal translation law. A model may have a modest average Bellman residual while a controller repeatedly chooses actions with poor realized increments; both quantities should therefore be logged.

22.3 Abel discovery advantage versus binary discovery gain

Section 21 defined finite-budget binary discovery gain $\text{DG}_R(a, x)$. An Abel coordinate provides a local graded analogue. Relative to a baseline action a_0 , define

$$\text{ADG}_A(a, x) := \mathbb{E}[\hat{A}(X_{t+1}) - \hat{A}(x) \mid x, a] - \mathbb{E}[\hat{A}(X_{t+1}) - \hat{A}(x) \mid x, a_0].$$

For unequal costs one should instead compare the cost-adjusted scores,

$$\widetilde{\text{ADG}}_A(a, x) = \hat{s}_a(x) - \hat{s}_{a_0}(x).$$

These quantities are not definitions of mathematical discovery and they are not automatically equal in sign to DG_R . A one-step drift advantage need not produce a larger R -step hitting probability when there are detours, state aliasing, heavy tails, or model error. Their value is experimental: they can distinguish two failed trajectories that both have binary score zero at budget R but move very differently through the search state space.

22.4 Approximation and action-gap stability

The practical question is not whether $\hat{A}$ is numerically exact. It is whether approximation error is small enough to preserve useful action decisions.

Proposition 22.2 (Action-gap stability under Abel-score error). *Fix a state x and let*

$$s_a(x) = \mathbb{E}[A^*(X_{t+1}) - A^*(x) \mid x, a] - c(x, a)$$

be the true Bellman-Abel score. Suppose

$$|\hat{s}_a(x) - s_a(x)| \leq \varepsilon$$

for every action. If the best true action a^ is unique and its gap over the second-best action satisfies*

$$s_{a^*}(x) - \max_{a \neq a^*} s_a(x) > 2\varepsilon,$$

then every maximizer of $\hat{s}_a(x)$ is a^ .*

Proof. For any $a \neq a^*$,

$$\hat{s}_{a^*} \geq s_{a^*} - \varepsilon > s_a + \varepsilon \geq \hat{s}_a.$$

□

This gives a concrete meaning to “good enough” Abel approximation. A small absolute error is not sufficient when competing interventions are nearly tied; a much larger error may be harmless when the action gap is large.

22.5 Positive Abel drift and discovery time

The settlement drift theorem from Section 13 applies directly when discovery rather than settlement is the absorbing target. Let $D_A(x) \geq 0$ be a shifted discovery-distance surrogate, for example $D_A = -A$ after normalizing the discovery set to level zero. If a frozen policy satisfies

$$\mathbb{E}[D_A(X_{t+1}) - D_A(X_t) \mid \mathcal{F}_t] \leq -\mu$$

with $\mu > 0$ before discovery, then under the same optional-stopping hypotheses,

$$\mathbb{E}[\tau_D] \leq \frac{D_A(X_0)}{\mu}.$$

Together with the finite-budget variance penalty already proved for Control AMLD, this suggests a discovery control signature containing at least

$$(\hat{\mu}, \hat{\sigma}^2, \rho_{BA}, \epsilon_A, h_R).$$

The binary hazard h_R says whether discovery occurs inside a declared horizon; Abel drift and residual say how the trajectory moves even when it does not.

22.6 Learning an approximate Abel coordinate without leakage

The forward-chaining requirement is strict. For rung k , $\hat{A}_k$ may use only trajectories from rungs $< k$. The model, normalization, action grammar, feature map, and hyperparameters are frozen before rung k is revealed to the verifier loop. Current-rung outcomes cannot be used to alter the coordinate that predicts the current rung.

Two complementary fitting losses are useful. On audited progress-bearing segments of earlier successful trajectories, a pathwise Abel loss is

$$\mathcal{L}_{\text{path}}(A) = \sum_{(x,a,x') \in \mathcal{T}_{\text{prog}}} w(x,a,x') (A(x') - A(x) - c(x,a))^2 + \lambda \sum_{z \in \mathcal{D}} A(z)^2.$$

The terminal term anchors discovery at zero. Because not every attempted transition is optimal or even productive, the pathwise loss should not be indiscriminately imposed on all actions.

A Bellman-style alternative uses all previously observed action outcomes:

$$\mathcal{L}_{BA}(A) = \sum_{x \in \mathcal{X}_{\text{train}}} \left[A(x) - \max_a \{ \hat{\mathbb{E}}[A(X') \mid x, a] - c(x,a) \} \right]^2 + \lambda \sum_{z \in \mathcal{D}} A(z)^2.$$

The first loss asks for translation along audited productive paths; the second asks for approximate dynamic-programming consistency. Agreement between them would be especially informative. Disagreement is also useful because it may reveal that a scalar coordinate is insufficient.

Caution 22.3 (Approximation is a hypothesis, not a theorem of theorem proving). Nothing above proves that a low-dimensional Abel coordinate exists for unrestricted proof search, that it transfers between mathematical domains, or that one scalar captures trajectories containing necessary detours. The approximation is justified only by held-out drift, residual, calibration, and control performance. If those fail, the correct response is to enlarge the state representation, use a vector or history-valued coordinate, or abandon the Abel layer rather than force the data into the equation.

The next section makes this falsifiability operational by treating each modification of the discovery controller as a new, frozen experimental iteration.

23 Experimental design for iterative discovery control

The discovery program is intentionally iterative. A failed intervention may reveal that the endpoint is too coarse, that the transformation grammar is weak, that the state representation omits a necessary variable, or that a promising idea simply does not transfer. Iteration is therefore not itself a methodological defect. The defect would be allowing information revealed by the current experiment to alter the current experiment while still reporting the result as if the protocol had been frozen in advance.

The governing principle is:

learn between experiments; freeze within experiments.

A completed iteration may motivate a new feature, action family, Abel loss, budget, or controller. That change receives a new protocol version and is tested prospectively. The previous result remains an immutable audit artifact.

23.1 Three statistical units: rung, trajectory, transition

The experiment has three nested units that should not be conflated.

1. A *rung* is a sealed theorem target with a fixed lower verified bank and declared representation interface.
2. A *trajectory* is one action-policy run on that rung under a fixed budget, seed, verifier, and candidate grammar.
3. A *transition* is one state-action-next-state observation inside a trajectory.

A run may generate many transitions, but these transitions are not independent replications of the theorem-level effect. Settlement and binary discovery comparisons should therefore be paired primarily by rung. Transition-level Abel analyses should preserve rung or trajectory clustering rather than treat every verifier call as an independent theorem.

23.2 The frozen forward-chain protocol

For rungs $1, 2, \dots, K$, let $\mathcal{H}_{k-1}$ be the complete audited history available before rung k . A leak-free iteration performs the following steps in order.

- E1. Declare the semantics.** Freeze the verifier, equivalence relation, weak/operational discovery criteria, cost metric, resource horizon R , and strict rejection rules for incomplete certificates.
- E2. Declare the observables.** Freeze the state feature map $\phi(X)$, including any residual-goal, bank, representation, motif, and controller-state summaries used by the model.
- E3. Declare the actions.** Freeze the intervention grammar. For the present program these are ordinary search, motif development, residual-goal repair, representation modulation, and counterpoint-style search.
- E4. Fit only on the past.** Train the finite-horizon hazard model, mechanism model, and any Abel/Bellman approximation using only $\mathcal{H}_{k-1}$.
- E5. Freeze the prediction.** Before the first verifier call on rung k , record the predicted

$$\hat{h}_a(X_k; R) = \widehat{\Pr}(\tau_{\mathcal{D}} \leq R \mid X_k, a),$$

the predicted Abel score $\hat{s}_a(X_k)$, and the selected action

$$\hat{a}_k \in \arg \max_a \hat{s}_a(X_k)$$

or the prospectively declared combined rule.

- E6. Run matched arms.** Give each intervention the same declared verifier-call budget, timeout, frozen lower bank, dependency environment, and seed schedule. Shared compiled sources may be cached, but their measured costs must be replay-accounted consistently.
- E7. Reveal and audit.** Only after all arms finish may the rung's outcomes, strict certificates, residual states, mechanism labels, and newly verified motifs enter the audit record.
- E8. Update the next iteration.** Add the audited rung to the training history for rung $k + 1$. Any change to the model class, feature map, action grammar, or budget is a new experiment version rather than a retroactive correction to rung k .

The strongest implementation records the pre-reveal predictions as a timestamped or hashed artifact before the verifier loop begins. This makes the no-peeking claim mechanically inspectable rather than dependent on memory.

23.3 The five-arm matched comparison

The current intervention set is:

- A_0 : ordinary search,
- A_1 : motif development,
- A_2 : residual-goal repair,
- A_3 : representation modulation,
- A_4 : counterpoint-style search.

The ordinary arm is the causal baseline. Every other arm must spend from the same budget R . Diagnostic tactics may inspect a goal and produce suggestions, but a suggestion is not a discovery until an explicit descendant source passes the strict verifier gate. Representation modulation may unfold, change basis, invoke a declared representation lemma, decompose, dualize, quotient, or normalize, but its transformation library must be frozen before the target is run. Counterpoint may exchange verified lemmas or diagnostic suggestions among coherent voices, but it receives no larger budget merely because it has more internal structure.

The primary theorem-level endpoint is

$$Y_R(a, k) = \mathbf{1}\{\text{strict discovery on rung } k \text{ within } R \text{ calls under } a\}.$$

The paired binary discovery gain is

$$\text{DG}_R(a, k) = Y_R(a, k) - Y_R(A_0, k)$$

for a deterministic matched replay, or the corresponding probability difference when repeated randomization supports probability estimation.

23.4 Primary and secondary measurements

A single success percentage is insufficient for an iterative control program. Each rung should record at least:

1. strict discovery/settlement within R ;
2. verifier calls and exact declared cost to discovery;
3. wall-clock verifier cost, kept separate from native proof cost;
4. predicted hazard and its calibration score;
5. predicted and realized Abel increments $\Delta\hat{A}_t$;
6. Bellman-Abel residual ρ_{BA} and trajectory residuals $\epsilon_{A,t}$;
7. increment variance and tail behavior;
8. residual-goal signature changes and representation changes;

9. strict rejects caused by ‘sorry’, ‘admit’, or equivalent incomplete-certificate conditions;
10. mechanism labels and motif/lemma transfer into later rungs;
11. action-selection regret relative to the per-rung hindsight oracle;
12. later operational usefulness U_R of any newly verified object.

For hazard calibration, the Brier score

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2$$

is a useful descriptive metric. For action choice, report both whether the selected action belongs to the hindsight argmax set and whether running only the prospectively selected action would have improved actual discovery rate or cost. A tie among all actions is not controller success merely because the selected action lies in the tied argmax set.

23.5 Iteration 1: the nine-rung H2 forward chain

The first explicit discovery forward-chain experiment used nine already solved H2-local targets, treated one at a time as if unknown. For rung k , only outcomes from rungs $< k$ were available to the predictor. Historical proof-search specifications and historical proof bodies were excluded, the current target’s canonical proof body was not read, and settlement required a strict Lean certificate with no ‘sorry’/‘admit’ source token or compiler warning. Each of the five actions received

$$R = 8$$

verifier calls with a 20-second per-call timeout.

The aggregate result was:

Action	Settled	Hit@1	Hit@2	Mean attempts on settled
Ordinary search	2/9	0.222	0.222	1
Motif development	2/9	0.000	0.000	3
Residual-goal repair	2/9	0.222	0.222	1
Representation modulation	2/9	0.111	0.111	4
Counterpoint search	2/9	0.222	0.222	1

Table 6: First leak-free H2 discovery forward chain at matched budget $R = 8$. All five arms reached exactly the same two of nine rungs.

The two reachable rungs were the permutation-sign-square and complement-round-trip targets. None of the five arms discovered any of the remaining seven within the budget. Consequently, on every rung and every nonbaseline action, the realized binary matched gain was

$$\text{DG}_8(a, k) = 0.$$

The per-rung hindsight oracle across all five arms therefore also reached only 2/9, exactly the ordinary-search reach. The frozen probability predictor had mean Brier score approximately 0.2218. Its selected action belonged to the binary hindsight argmax set on 9/9 rungs, but that statistic is vacuous here because the binary argmax set contained every action on every rung.

This is a useful negative control. It does not show that motif development, repair, modulation, or counterpoint are intrinsically useless. It shows that the implemented grammars did not enlarge the strict $R = 8$ reachable set, and that the binary endpoint supplied no differential training signal for choosing among them. The experiment therefore motivates a finer state-transition measurement rather than post hoc tuning of an exploration constant.

23.6 Iteration 2: Discovery Abel v4

The next experiment should preserve the same strict certificate gate and matched-budget logic while adding transition-level observation. Every verifier call should record the complete reproducible state summary before the action, the chosen intervention and candidate family, the next residual state, the exact declared cost, the strict verifier result, and the features required to evaluate $\hat{A}$.

For rung k , fit $\hat{A}_k$ only on earlier rungs. Before revealing any current-rung outcome, freeze:

$$\hat{h}_a(X_k; R), \quad \hat{\mu}_a(X_k), \quad \hat{s}_a(X_k), \quad \hat{a}_k.$$

Then run the same five arms. The principal new questions are:

- (a) Does positive held-out Abel drift predict later strict discovery even when $Y_R = 0$ at the current horizon?
- (b) Does the Bellman-Abel residual decrease as the training history grows?
- (c) Does the intervention with largest predicted Abel score produce larger realized drift than ordinary search on unseen rungs?
- (d) Does selecting the largest predicted drift eventually improve binary discovery probability, cost-to-discovery, or both?
- (e) Are the gains robust when the target requires a temporary detour that makes a naive distance coordinate worse before it improves?

The primary causal comparison remains the prospectively selected controller versus ordinary search. The four individual nonbaseline arms are mechanistic ablations. This keeps multiple exploratory comparisons from being mistaken for four separate confirmatory claims.

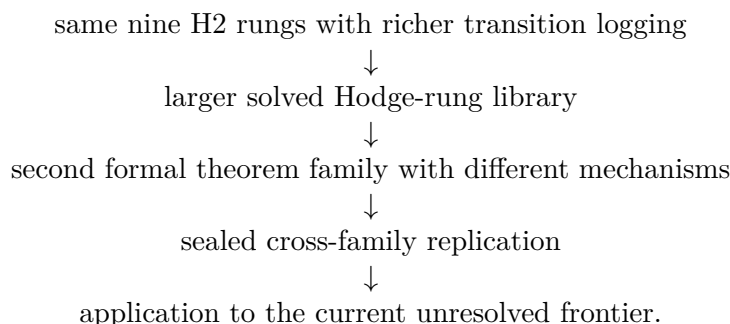
23.7 Analysis rules for small forward chains

With nine rungs, raw paired outcomes and uncertainty are more informative than elaborate significance claims. For binary outcomes, report the full 2×2 paired table against ordinary search and use an exact paired test only when discordant rungs exist. For continuous or ordinal costs, report paired rung-level differences and robust intervals. For Abel transition data, do not compute artificially tiny standard errors by treating hundreds of transitions from one rung as hundreds of independent theorem targets; resampling or uncertainty estimates should cluster at least by rung and preferably by theorem family.

Model comparison must also remain forward. Hyperparameters may be selected from earlier rungs or from an independent development family, but not by observing which setting would have won on the current sealed rung. If a new loss, normalization, motif grammar, or feature set is proposed after examining the current results, that proposal becomes the next protocol version.

23.8 Escalation across theorem families

A discovery controller should not be trusted on a live frontier merely because it works on one local H2 sequence. A reasonable escalation ladder is:



At every stage the current target remains hidden from the model used to predict it. Cross-family replication is especially important for testing whether the Abel coordinate captures a transferable property of discovery dynamics rather than memorizing one local sequence of Hodge constructions.

23.9 Decision rules for the next iteration

The iterative program should have explicit reactions to possible outcomes.

- If binary discovery gain is positive and replicated, retain the successful intervention and test whether the gain survives new families and budgets.
- If binary gain is zero but Abel drift separates the arms and predicts later discovery, extend the horizon or strengthen the controller while keeping the graded signal frozen prospectively.
- If Abel drift separates arms but does not predict later discovery, treat it as a descriptive coordinate rather than a control variable.
- If no scalar coordinate predicts either progress or intervention benefit, test a vector or history-dependent representation instead of forcing a scalar Abel model.
- If an apparent gain disappears under the strict certificate gate, preserve the diagnostic trace but reject the mathematical conclusion.
- If a new representation or mechanism is invented after seeing a failure, document it as the hypothesis of the next iteration rather than rewriting the failed iteration as a success.

This versioned procedure turns repeated experimentation into cumulative evidence. The machine is allowed to learn from failure; the experiment is not allowed to forget that the failure happened.

24 Conclusion

The central proposal is that theorem proving and conjecture settlement can be studied as exact rationally weighted maze solving over verifier-backed state graphs. The internal search process may be nonmonotone, stochastic, learned, or self-reflective, while its cumulative history of verified information forms a monotone SIC. Under additional target-search conditions that SIC is an ATP; with typed proof/refutation/independence certificates it becomes an ALD; one logical level higher it becomes an AMLD.

The graph-theoretic perspective matters because it imports a large body of route-finding ideas subject to explicit hypotheses. For finite nonnegative rational weights, Dijkstra-style minimum-cost search is exact. With admissible lower bounds, A*-style and branch-and-bound pruning can reduce the region below an incumbent. Novelty DAGs eliminate verified-state cycles, conservative compilation replaces repeated verified paths by macro edges, and shared lemma banks turn repeated proof structure into reusable landmarks. These reductions do not solve undecidability or guarantee practical access to every fixed point. They narrow the ocean in which the machine must search.

The discovery layer adds a different measurable question. A search may reach a certificate without learning a reusable mechanism, or it may first acquire a verified novelty that changes what later search can reach or how cheaply it can settle targets. By separating weak discovery from operational discovery, and by measuring discovery hazard and intervention gain on sealed earlier rungs, the framework makes this distinction experimentally testable. A forward-chaining controller may learn from previous rung trajectories while remaining frozen with respect to the rung it is currently attempting.

The Abel layer addresses the cases in which the strict binary endpoint is scientifically clean but statistically coarse. When optimal time-to-discovery satisfies Bellman recursion, its negative is an Abel coordinate: under unit cost an optimal pre-discovery action advances expected Abel position by one, while exact rational transaction costs produce the corresponding cost-adjusted translation law. In realistic search this coordinate must be approximated. The relevant questions are therefore whether held-out Abel drift is positive, whether Bellman-Abel residual and variance remain controlled, whether approximation error is small relative to the action gap, and whether the action chosen by the approximate coordinate improves future verifier-backed discovery rather than merely correlating with retrospective success.

The first leak-free nine-rung H2 discovery experiment is deliberately retained as a negative control. Ordinary search, motif development, residual-goal repair, representation modulation, and counterpoint search all reached exactly the same two of nine targets at $R = 8$. Thus binary matched discovery gain was zero for every nonbaseline arm, and a hindsight oracle over the five arms could not exceed the ordinary-search reachable set. That failure is informative: it says that the next iteration should record graded state transitions and test an Abel coordinate, not rescue the result by tuning a global exploration parameter after the fact.

The experimental-design principle is therefore cumulative but strict: learn between experiments and freeze within experiments. A completed run may suggest a new state feature, representation move, motif grammar, loss function, or controller; the change receives a new protocol version. Pre-reveal predictions, verifier gates, budgets, dependencies, seeds, and treatment grammars are preserved so that each rung can be audited as a prospective test. Transition-level data are clustered within rungs rather than counted as independent theorem replications, and a controller is promoted toward a live frontier only after sealed cross-family evidence supports transfer.

The motif-semigroup picture adds a disciplined vocabulary for structured variation. Instead of injecting a fixed fraction of noise, a controller can develop, transpose, invert, combine, quotient, decompose, or modulate a proof motif when earlier controlled experiments indicate that such a mechanism has positive discovery or Abel advantage in a comparable state. The transformed proposal remains untrusted until verification. This keeps creativity, discovery, functional iteration, and control inside the same certificate-first architecture as the rest of the paper.

The further reflective possibility is that the induced ATP can itself reason about a coded source ATP, ALD, AMLD, or controller. If such meta-theorems can be verified and fed back into search, the machine does not merely learn statistically from its trajectories; it can sometimes use proved facts about its own frozen search architecture. DATA is a distinguished laboratory for this general theory, not its definition.

References

- [1] Edward F. Moore. *The Shortest Path Through a Maze*. In *Proceedings of the International Symposium on the Theory of Switching, Part II*, pp. 285–292. Harvard University Press, 1959.
- [2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. <https://doi.org/10.1007/BF01386390>
- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. <https://doi.org/10.1109/TSSC.1968.300136>
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, 4th ed. MIT Press, 2022. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- [5] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020. <https://aima.cs.berkeley.edu/>
- [6] Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962. <https://doi.org/10.1145/367766.368168>
- [7] Stephen Warshall. A theorem on Boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962. <https://doi.org/10.1145/321105.321107>
- [8] Antti Valmari. A stubborn attack on state explosion. *Formal Methods in System Design*, 1(4):297–322, 1992. <https://doi.org/10.1007/BF00709154>
- [9] Doron Peled. All from one, one for all: on model checking using representatives. In *Computer Aided Verification (CAV 1993)*, Lecture Notes in Computer Science 697, pp. 409–423. Springer, 1993. https://doi.org/10.1007/3-540-56922-7_34
- [10] Alice Miller, Alastair F. Donaldson, and Muffy Calder. Symmetry in temporal logic model checking. *ACM Computing Surveys*, 38(3), 2006. <https://doi.org/10.1145/1132960.1132962>
- [11] Richard E. Korf and Ariel Felner. Disjoint pattern database heuristics. *Artificial Intelligence*, 134(1–2):9–22, 2002. [https://doi.org/10.1016/S0004-3702\(01\)00092-3](https://doi.org/10.1016/S0004-3702(01)00092-3)
- [12] Michel Gondran and Michel Minoux. *Graphs, Dioids and Semirings: New Models and Algorithms*. Springer, 2008. <https://doi.org/10.1007/978-0-387-75450-5>
- [13] Brian Tenneson. *A Closed-Form Surjection from N onto Q* . Version 2.2, 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/math.L0_Logic/A%20Closed-Form%20Surjection%20from%20N%20onto%20Q%20by%20Brian%20Tenneson%202.2.pdf
- [14] Brian Tenneson. *Depths of the theorem-search program: graph-theoretic proof search, SICs, and automated theorem proving*. Public 2026 research manuscript in the *Depths* line; live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.L0_Logic_in_Computer_Science/Depths_of_Induction_ML_Guided_Proof_Search_and_Formalized_Self_Awareness_Version_47.pdf

- [15] Brian Tenneson. *The Universal Proof-Horizon Operator: From MIU and the n th Well-Formed Formula to ZFC, Predator, and Automated Logical Deciders*. Version 1.0, August 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.L0_Logic_in_Computer_Science/The_Universal_Proof-Horizon_Operator_by_Brian_Tenneson.pdf
- [16] Brian Tenneson. *DATA 2.0.1: Proof-Horizon Architecture / From a Target Formula to a Verified Minimum-Cost Proof*. August 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.L0_Logic_in_Computer_Science/DATA_2_0_1_Proof_Horizon_Architecture_FINAL.pdf
- [17] Brian Tenneson. *A Technical Companion to My Next Generation Automated Theorem Prover*. 2026; includes the Proof-Horizon/Ocean synthetic benchmark material used here. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.L0_Logic_in_Computer_Science/A_Technical_Companion_to_My_Next_Generation_Automated_Theorem_Prover_v1.0.pdf
- [18] Brian Tenneson. *Hilbert Space-Filling Curve Theorem Search*. Version 0.5.3, 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.L0_Logic_in_Computer_Science/Hilbert_Space_Filling_Curve_Theorem_Search_v0.5.3.pdf
- [19] Brian Tenneson. *DATA 3.0: AMLD/IFS Semigroup Architecture, Settlement Horizons, and Conjecture-Settling Research Program*. August 2026. Live GitHub research library: https://github.com/btenneson/pub/tree/main/cs.L0_Logic_in_Computer_Science
- [20] Marek Kuczma, Bogdan Choczewski, and Roman Ger. *Iterative Functional Equations*. Encyclopedia of Mathematics and its Applications, Vol. 32. Cambridge University Press, 1990. See especially Chapter 9, “More on the Schroeder and Abel equations.” <https://doi.org/10.1017/CB09781139086639>
- [21] Kurt Goedel. Ueber formal unentscheidbare Saetze der Principia Mathematica und verwandter Systeme I. *Monatshefte fuer Mathematik und Physik*, 38:173–198, 1931. <https://doi.org/10.1007/BF01700692>
- [22] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936. <https://doi.org/10.2307/2371045>
- [23] Alfred Tarski. Der Wahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica*, 1:261–405, 1935.